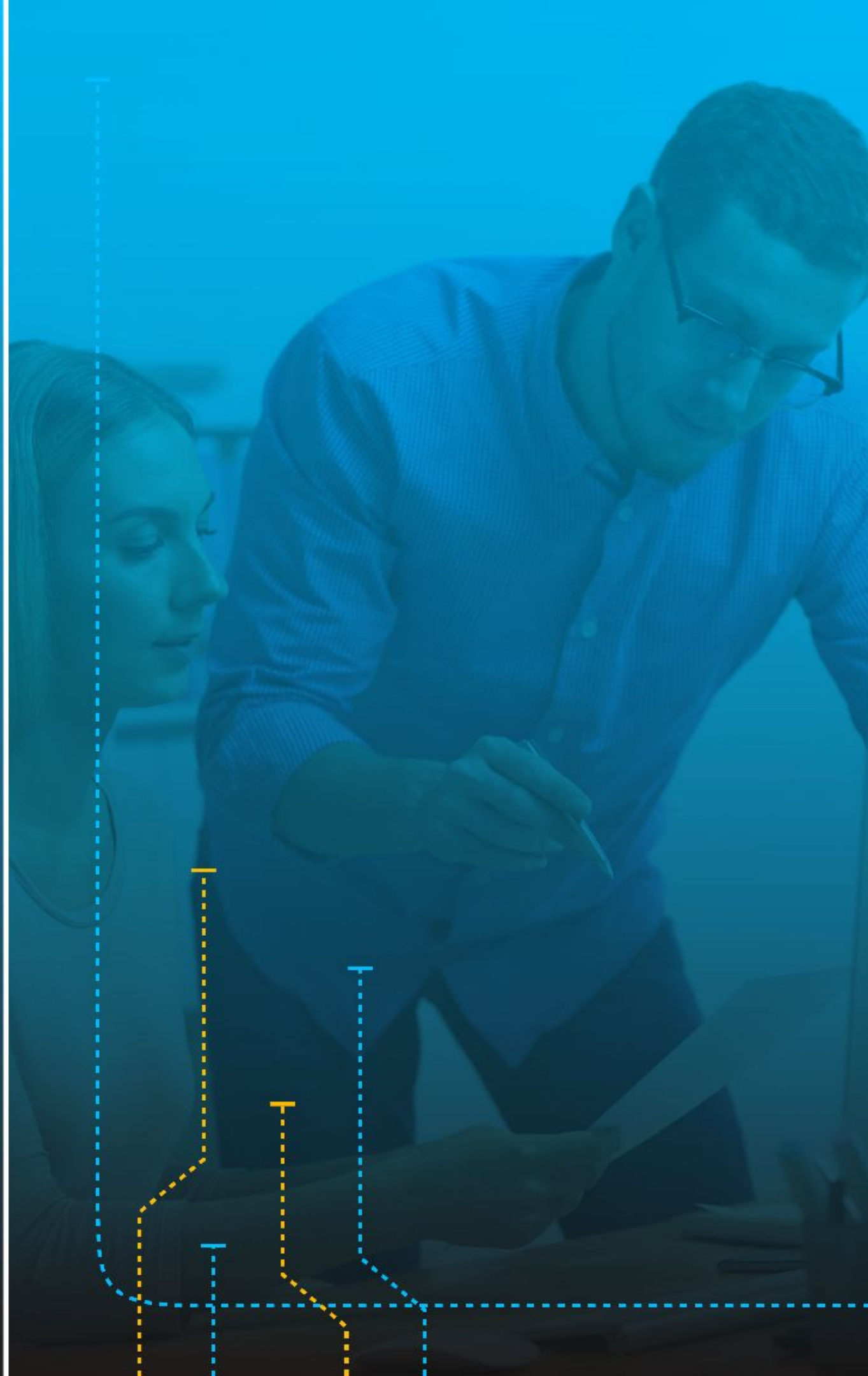


Internet de las Cosas

LAB 1 – Arduino

Introducción a Arduino + Instalación

Sesión de Laboratorio



En esta experiencia de laboratorio, exploraremos diversas prácticas orientadas al aprendizaje activo de los principios fundamentales de los circuitos eléctricos y su simulación. A lo largo de las sesiones, los estudiantes desarrollarán competencias prácticas mediante el uso de plataformas digitales y componentes reales. Aquí se presentan los objetivos principales formulados de manera clara y didáctica:

1. **Simulación de Circuitos.** Utilizar Tinkercad y Falstad para construir y analizar circuitos eléctricos de forma virtual.
2. **Conexiones de Resistencias.** Aplicar configuraciones en serie y paralelo para calcular la resistencia equivalente.
3. **Montaje de Componentes Básicos.** Integrar resistencias, LEDs, potenciómetros y botones en circuitos prácticos.
4. **Diseño de un Semáforo.** Implementar un sistema de control de semáforo con lógica programada y entradas digitales.
5. **Uso de Instrumentos de Medición.** Medir voltaje, corriente y resistencia con instrumentos adecuados y conexiones correctas.

1.

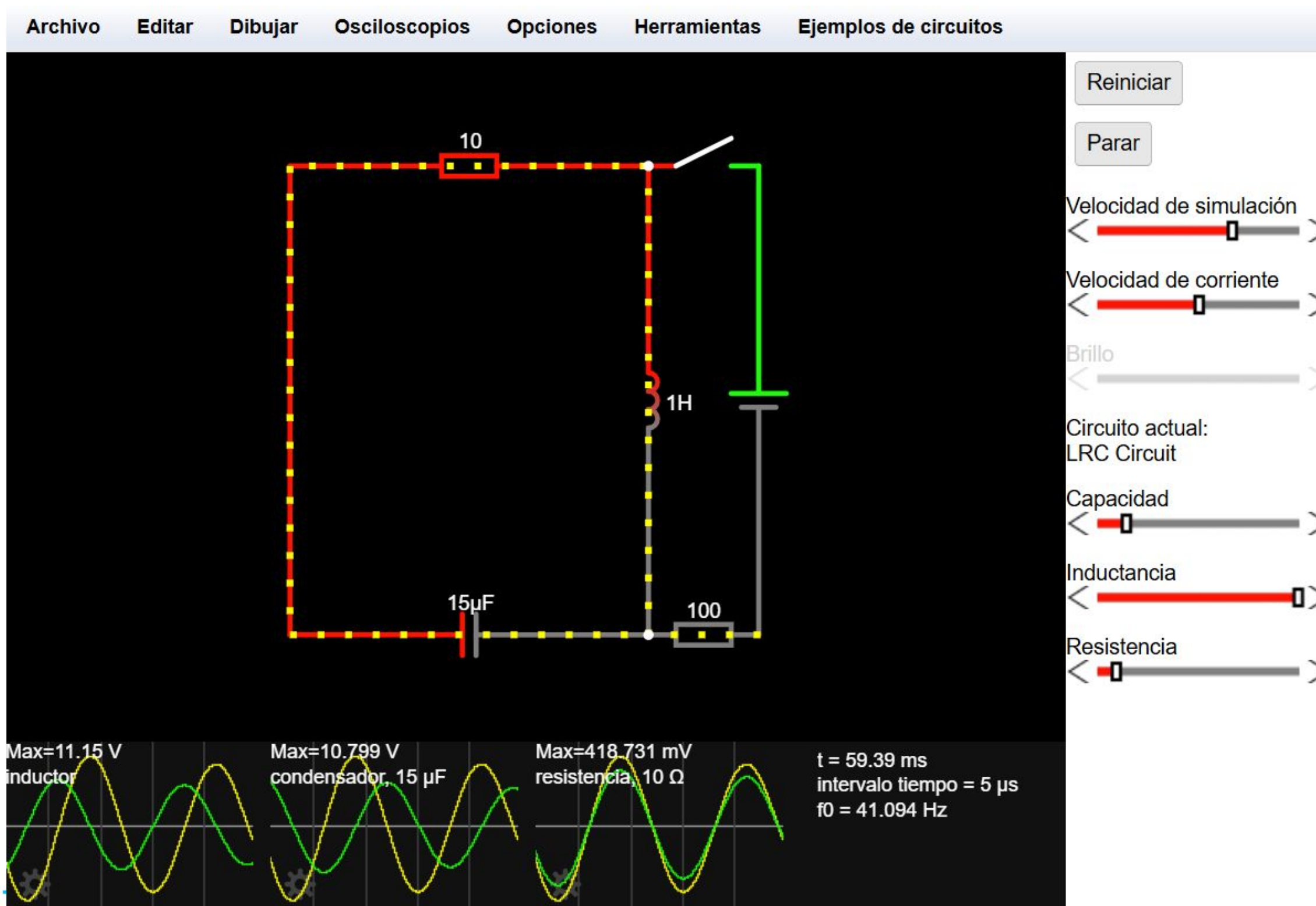


Simulación de Circuitos Eléctricos

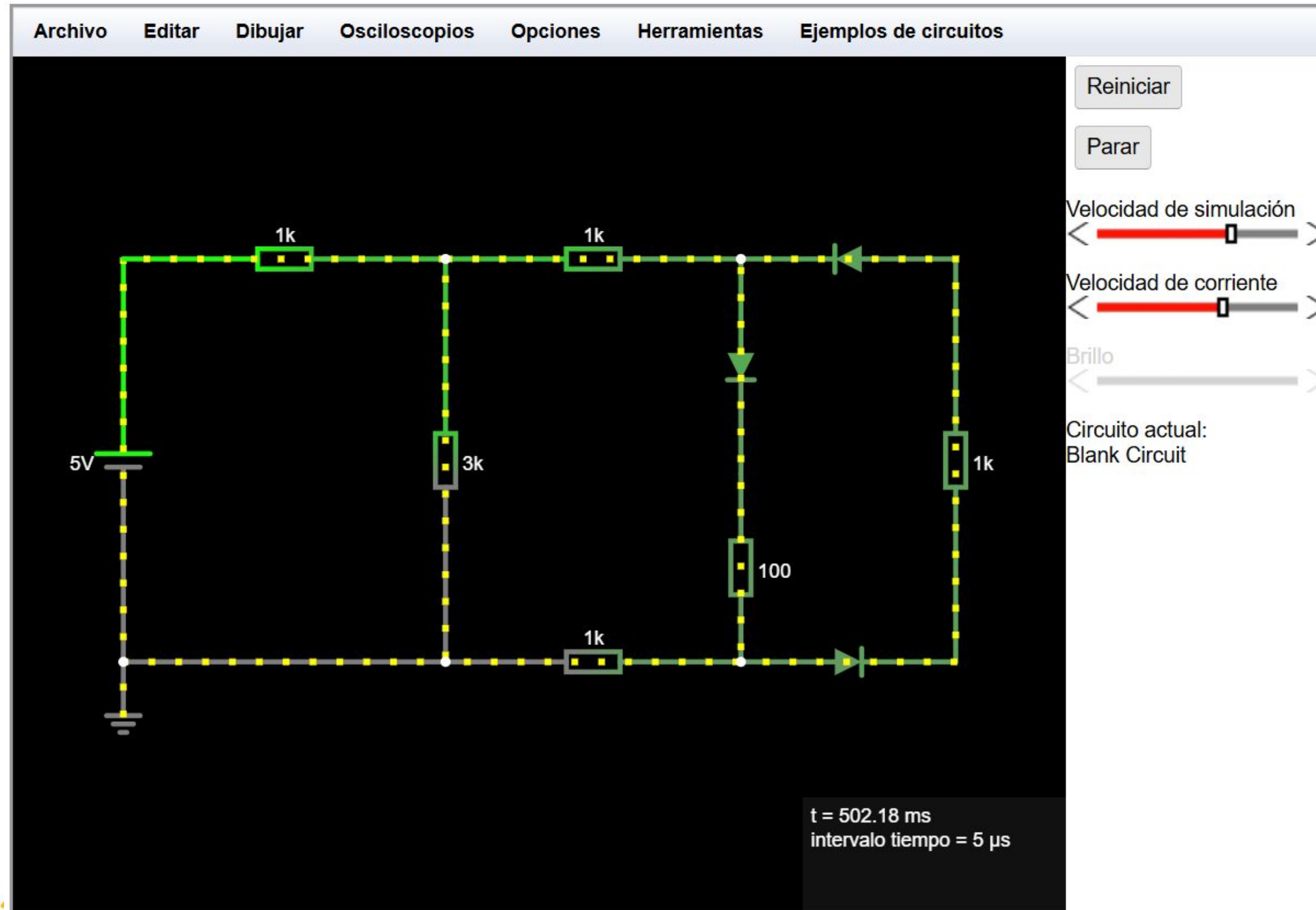
Tinkercad y Falstad

FALSTAD

[HTTPS://FALSTAD.COM/CIRCUIT/](https://falstad.com/circuit/)



FALSTAD [HTTPS://FALSTAD.COM/CIRCUIT/](https://falstad.com/circuit/)



```
<cir f="1" ts="0.000005"
ic="10.20027730826997" cb="62" pb="50" vr="5"
mts="5e-11">
  <v x="160 384 160 144" f="16" wf="0"
maxv="5"/>
  <r x="160 144 352 144" f="0" r="1000"/>
  <r x="352 144 352 384" f="0" r="3000"/>
  <w x="352 384 160 384" f="0"/>
  <r x="352 144 528 144" f="0" r="1000"/>
  <r x="528 384 352 384" f="0" r="1000"/>
  <d x="656 144 528 144" f="0" mo="default"/>
  <r x="656 144 656 384" f="0" r="1000"/>
  <d x="528 384 656 384" f="0" mo="default"/>
  <g x="160 384 160 416" f="0"/>
  <d x="528 144 528 272" f="0" mo="default"/>
  <r x="528 272 528 384" f="0" r="100"/>
</cir>
```

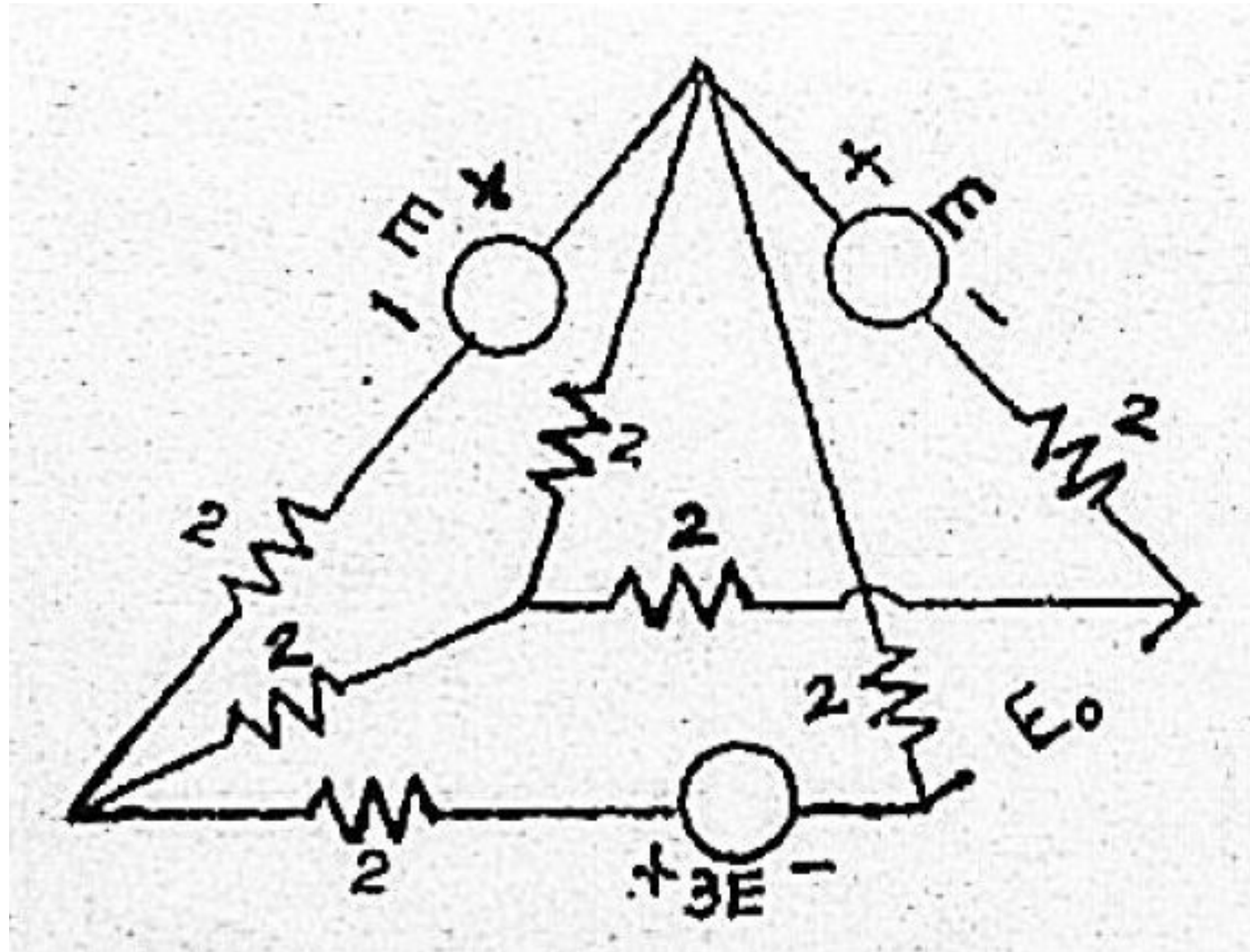
ENLACE AL
CIRCUITO

TRANSFORMATEC

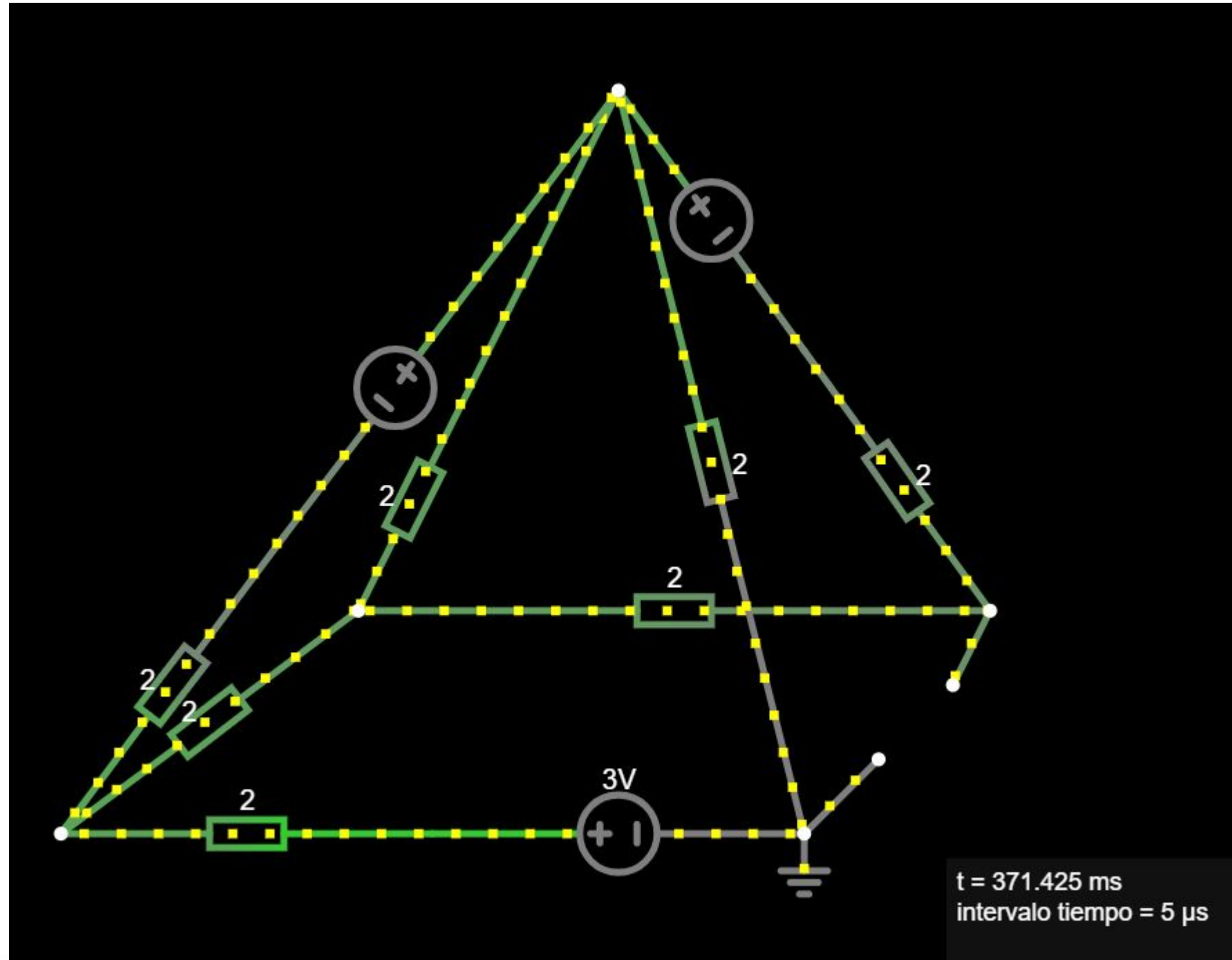
FALSTAD [HTTPS://FALSTAD.COM/CIRCUIT/](https://falstad.com/circuit/)

7. Halle (Con $E = 1V$)

- (a) Determinar el valor de E_0 .
- (b) Calcule el valor de la potencia entregada por la fuente $3E$.
- (c) Cuál es el valor de la corriente que pasa por E_0 .

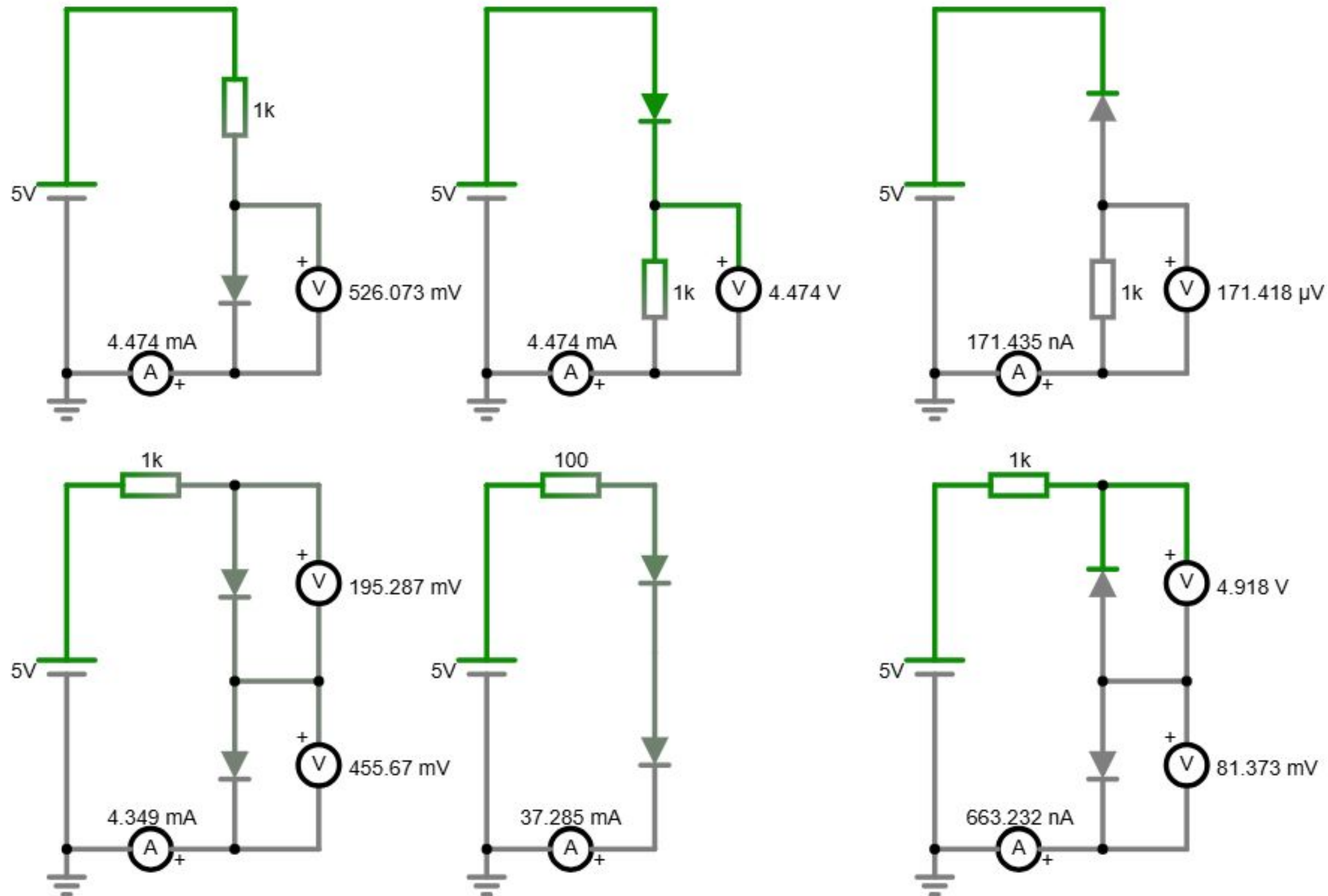


FALSTAD [HTTPS://FALSTAD.COM/CIRCUIT/](https://falstad.com/circuit/)



ENLACE AL
CIRCUITO

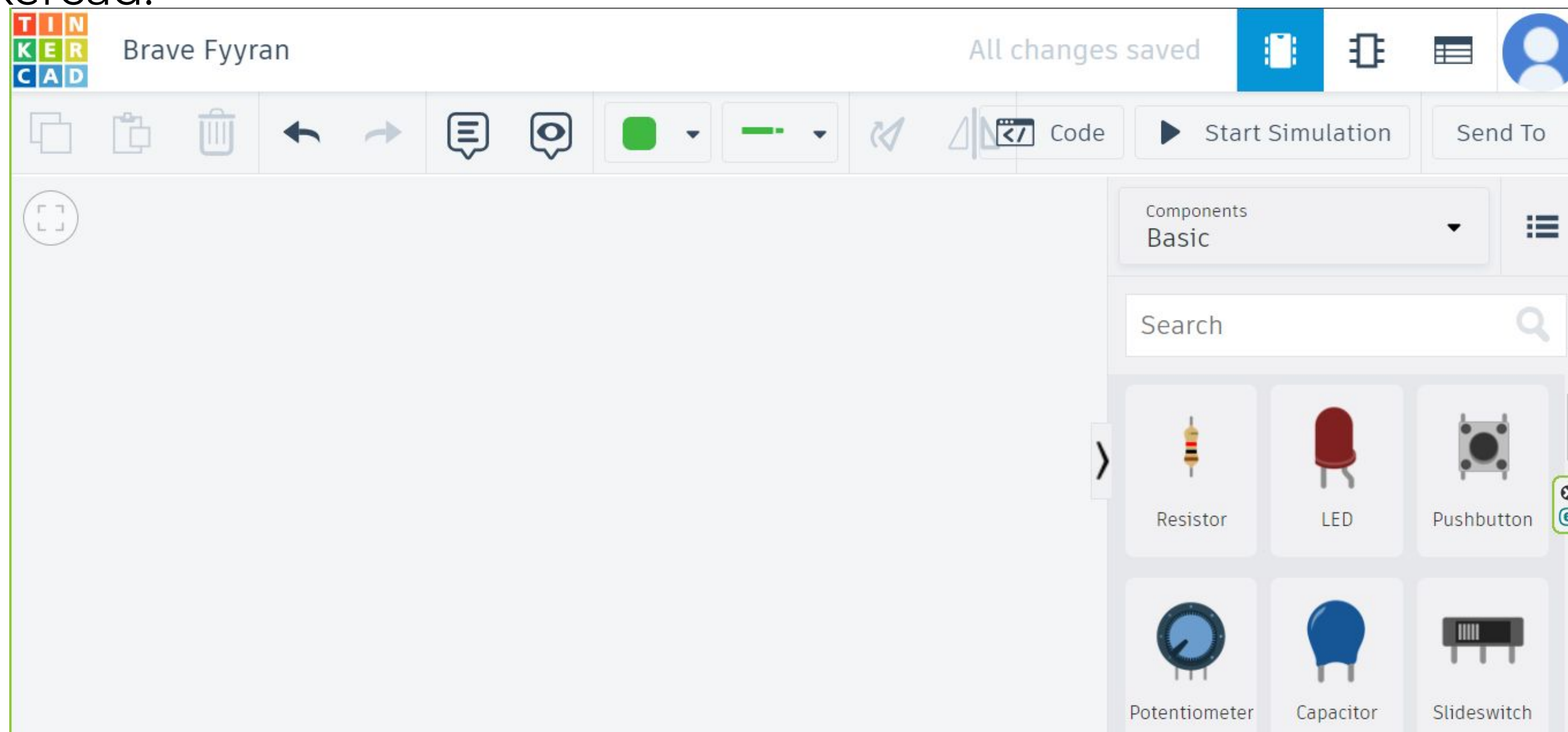
FALSTAD [HTTPS://FALSTAD.COM/CIRCUIT/](https://falstad.com/circuit/)



**ENLACE AL
CIRCUITO**

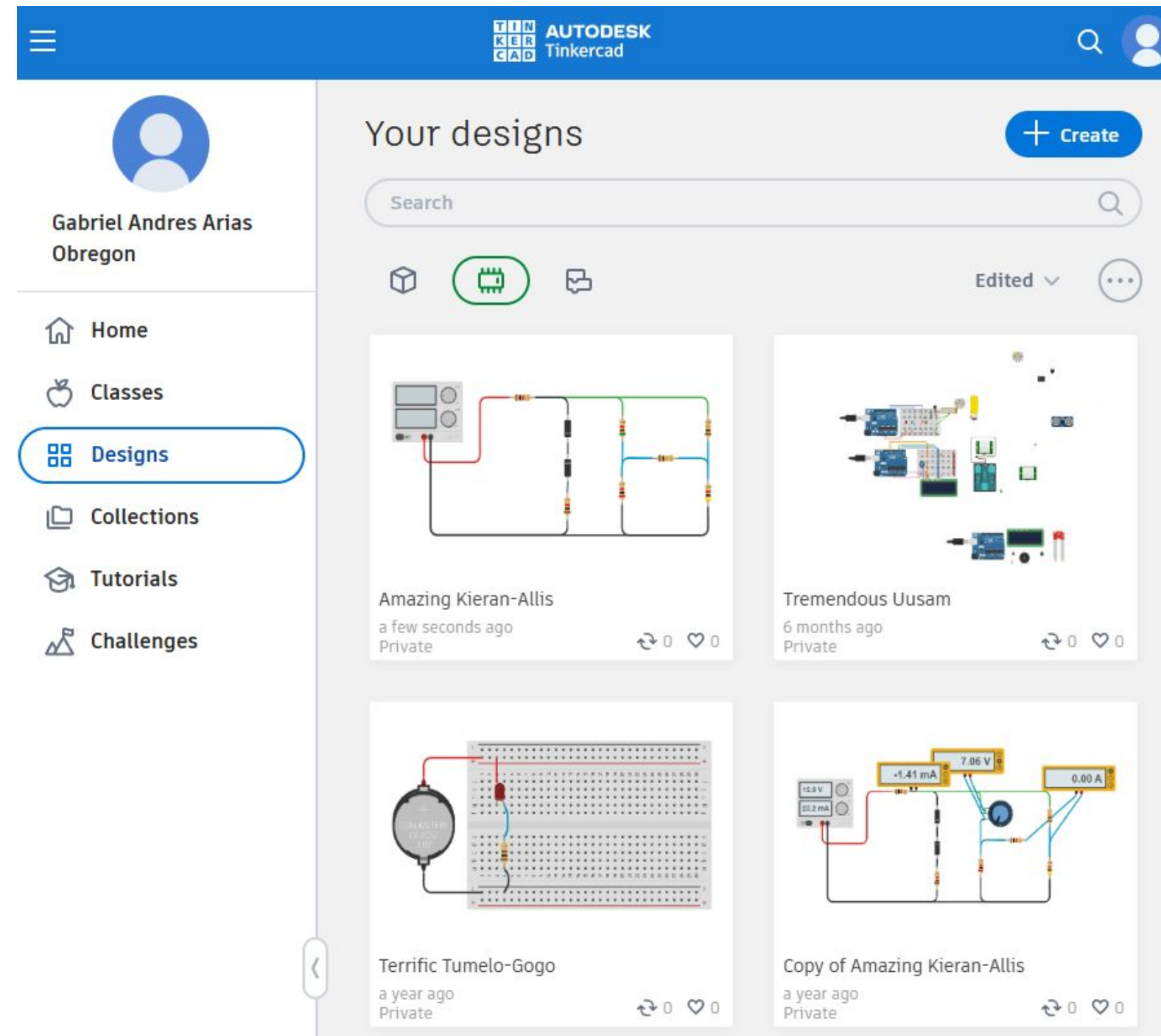
Espacio de Trabajo

En Tinkercad.



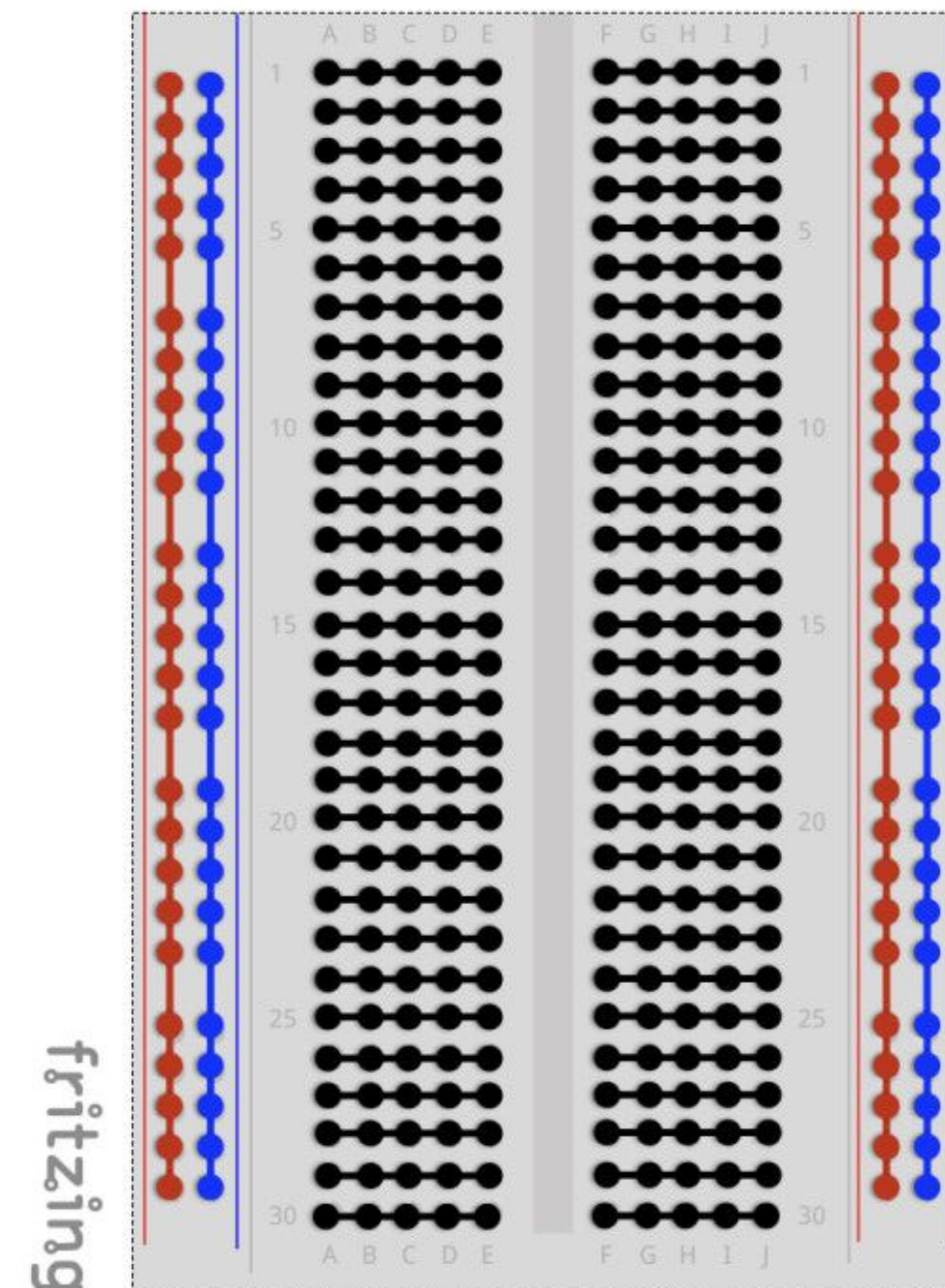
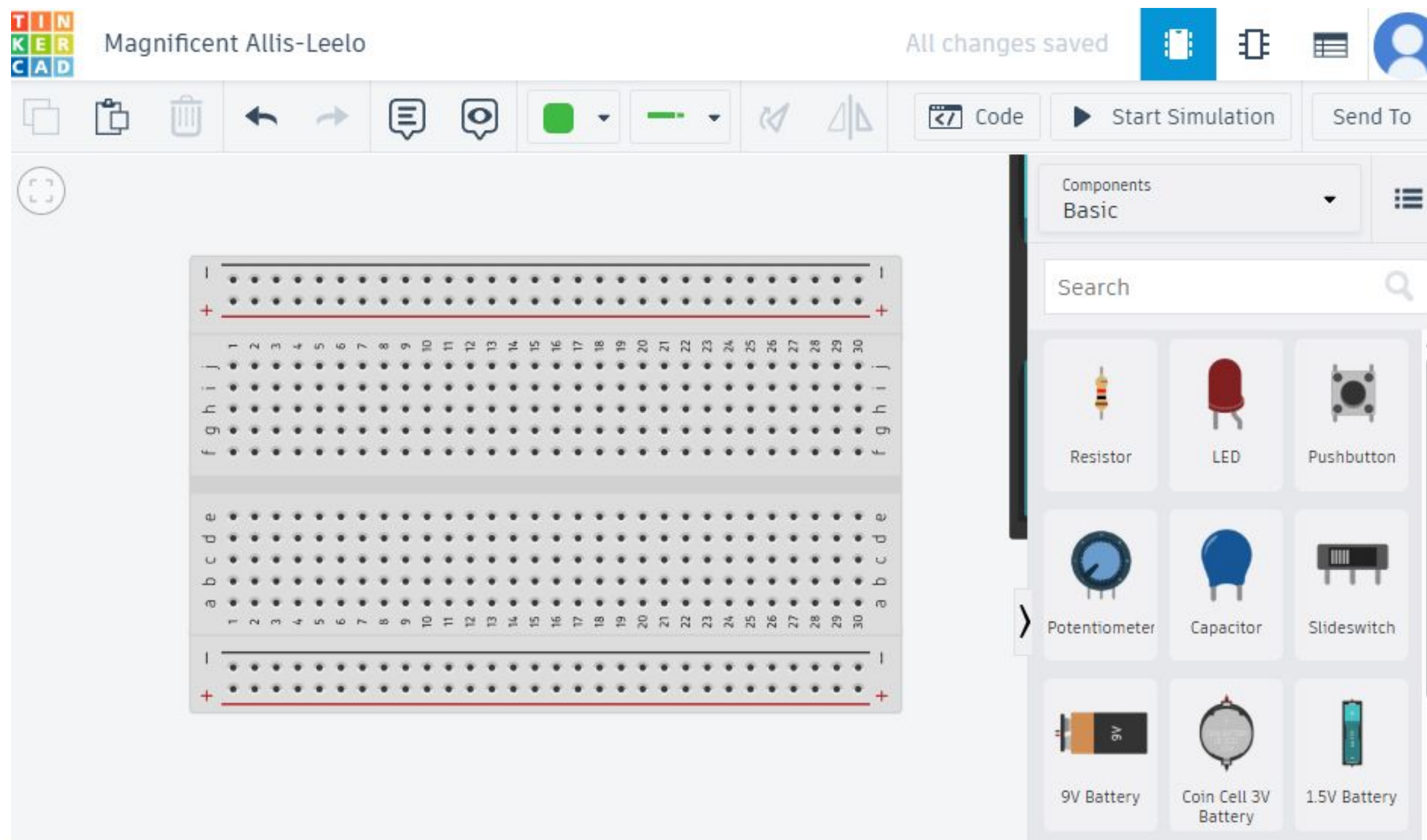
Espacio de Trabajo

En Tinkercad.



Espacio de Trabajo

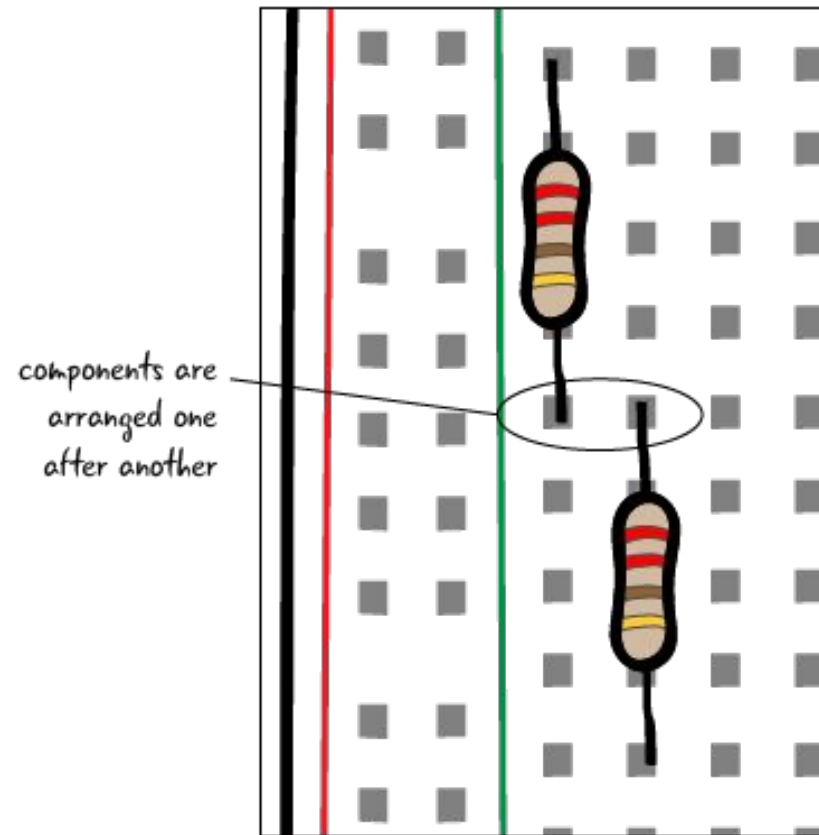
En Tinkercad.



Conexión de Resistencias

En protoboard.

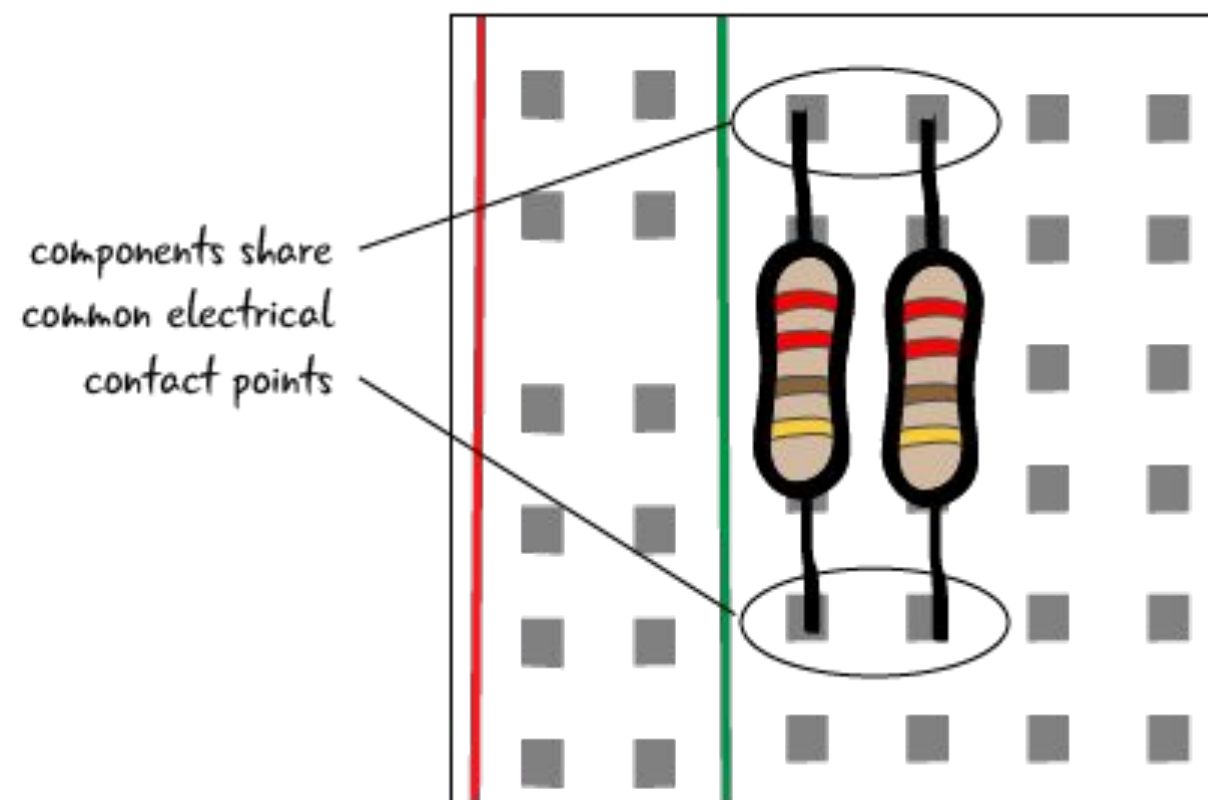
resistors arranged in series



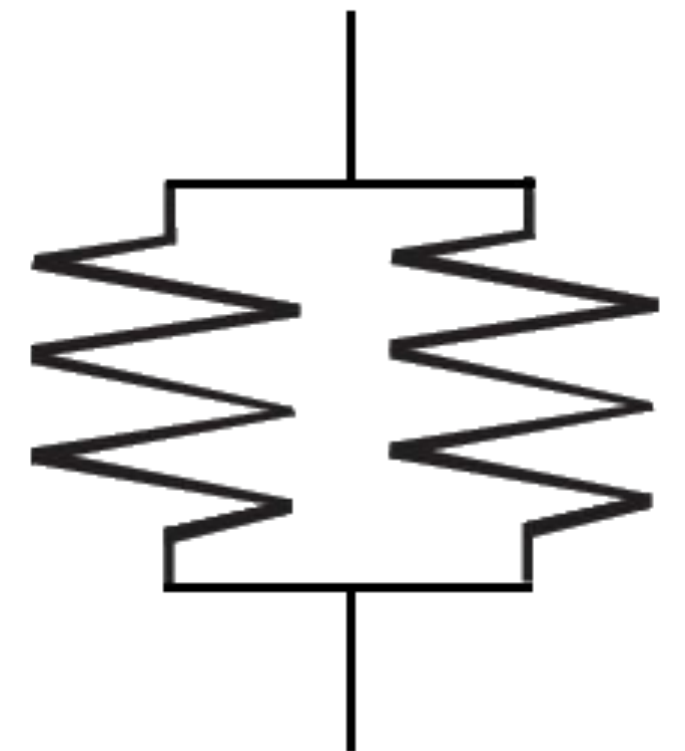
schematic of resistors arranged in series



resistors arranged in parallel

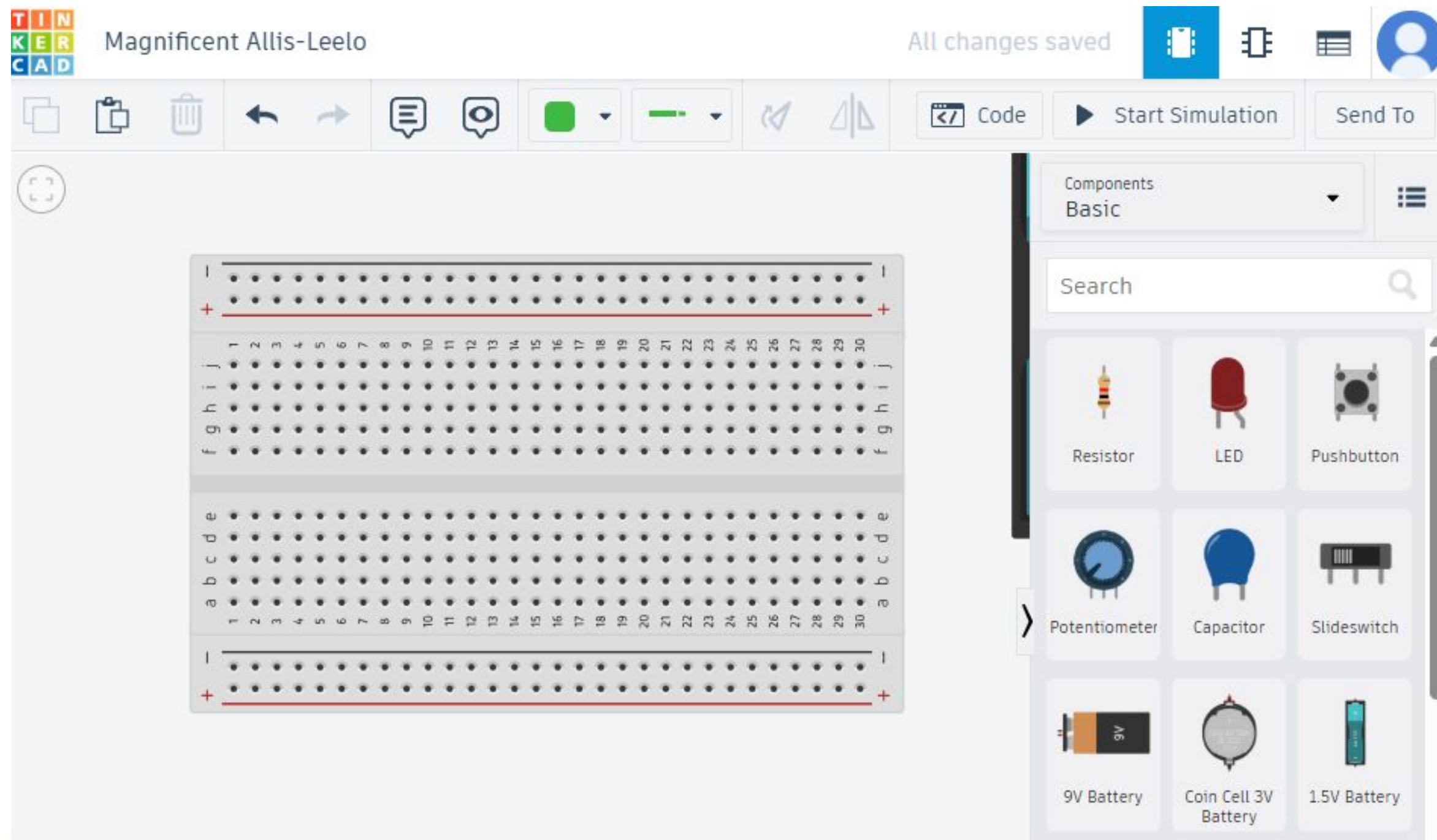


schematic of resistors arranged in parallel

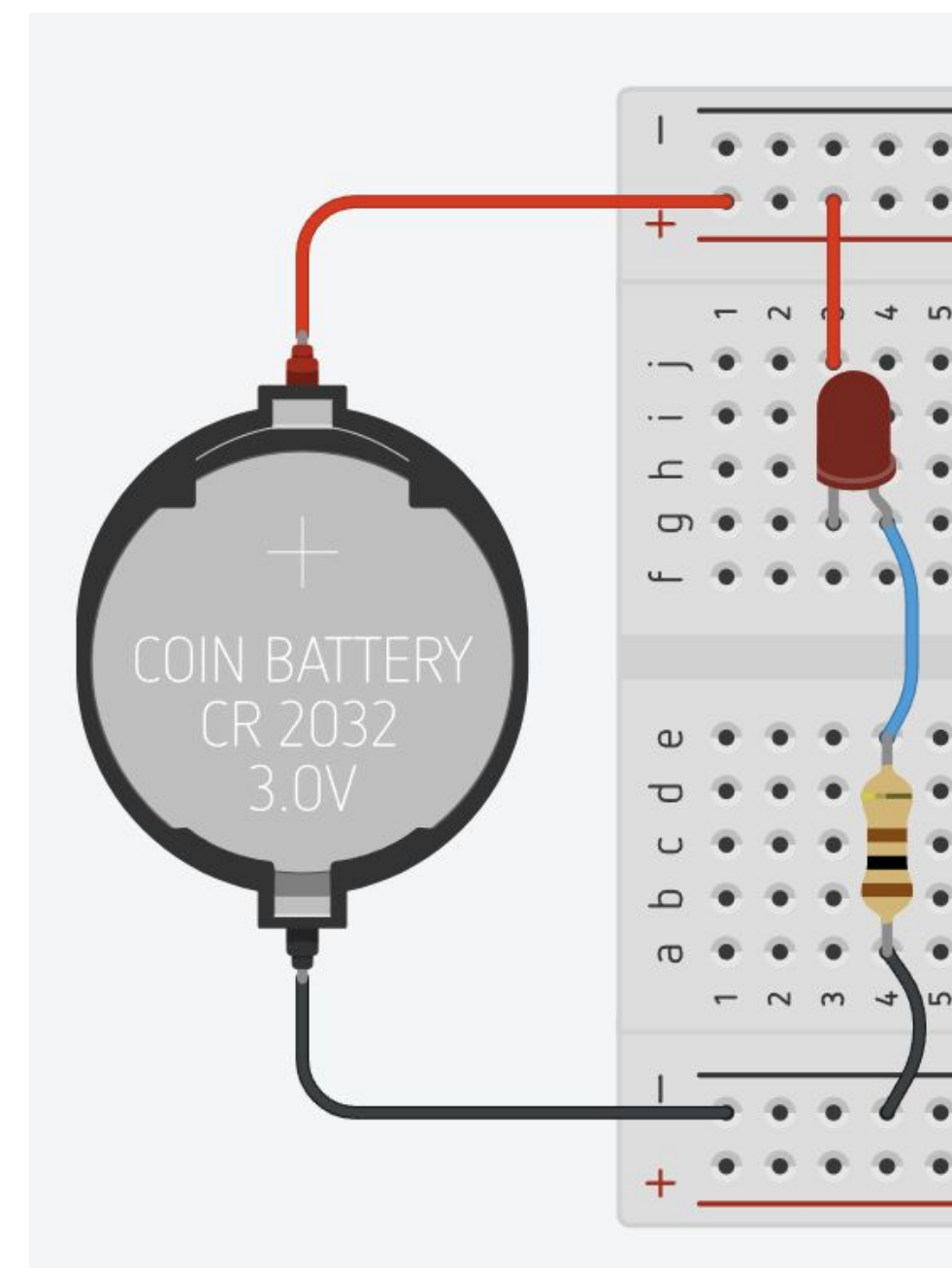


Espacio de Trabajo

En Tinkercad.



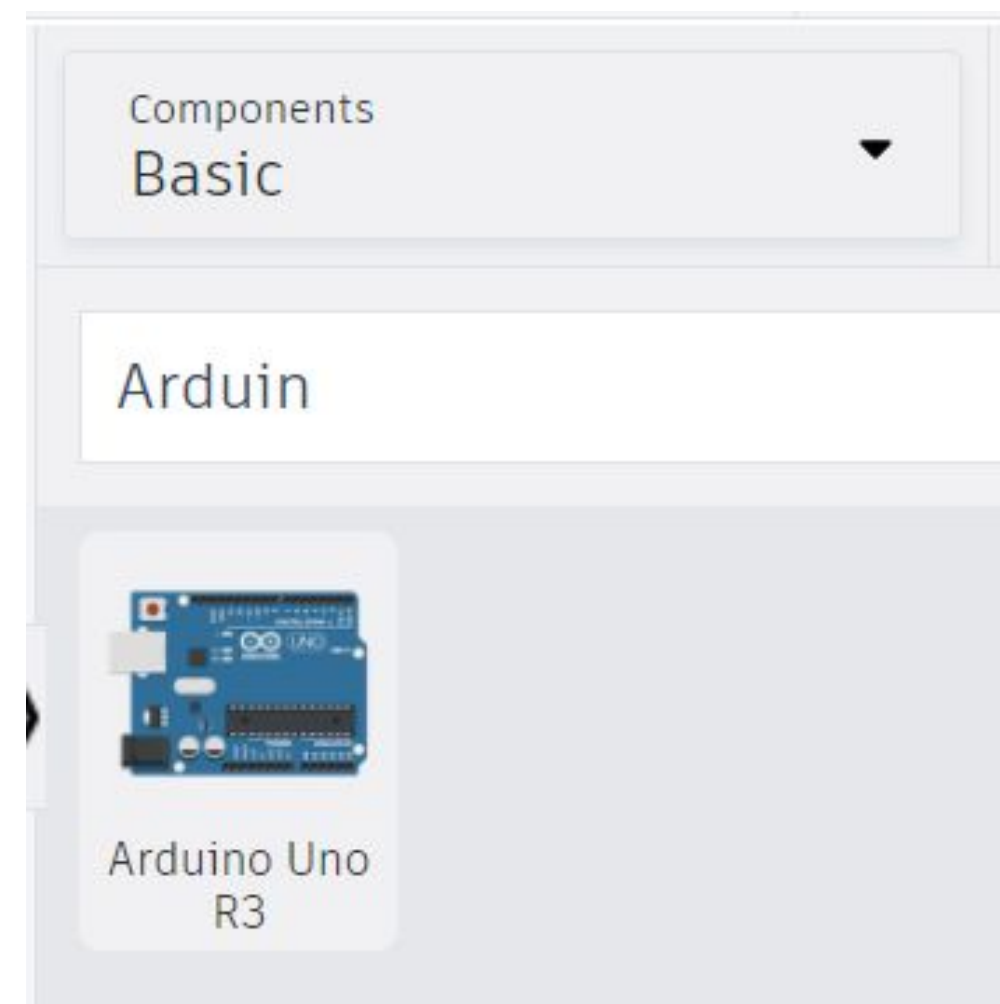
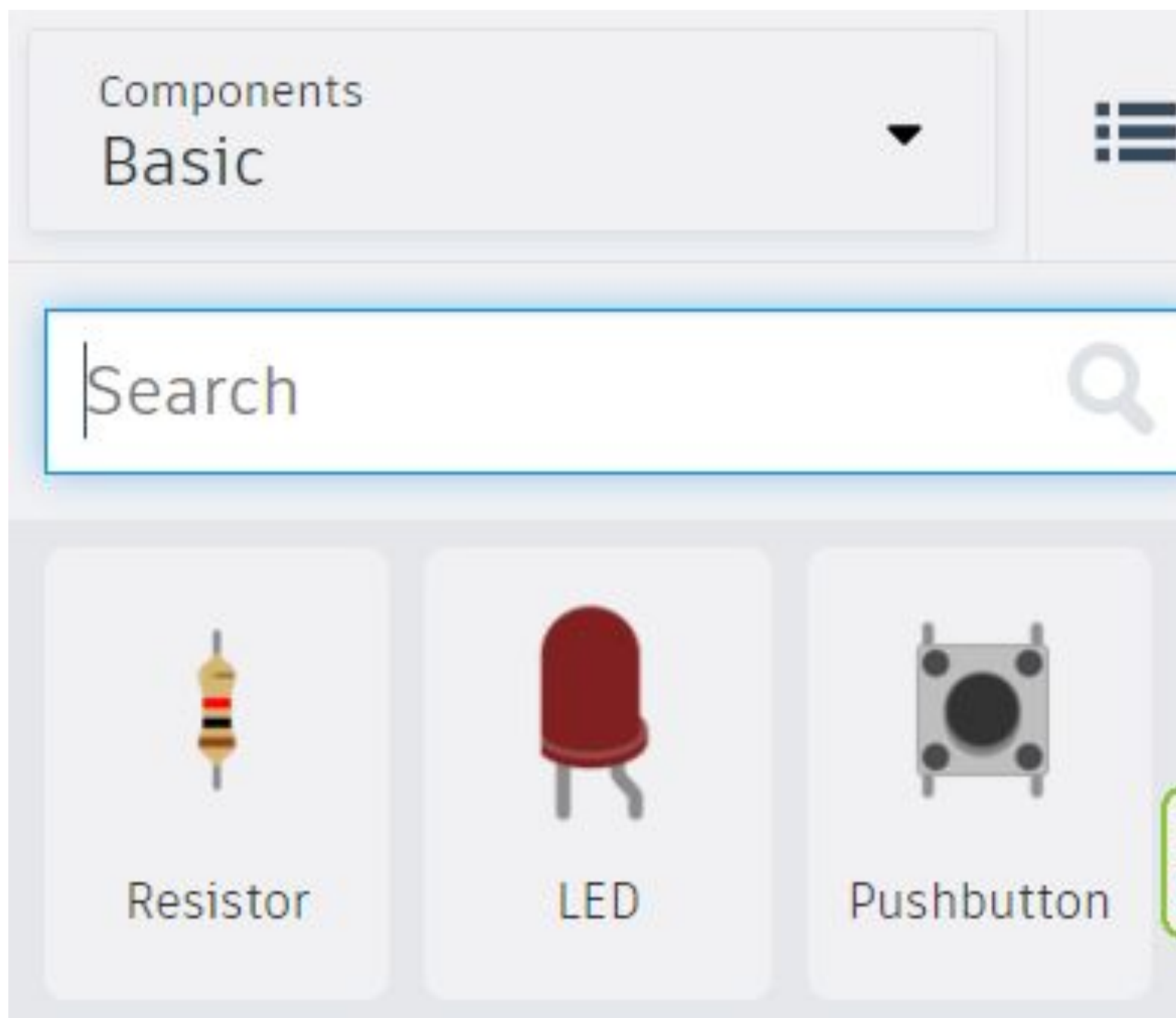
> Reinventa el mundo <



¿Funciona?

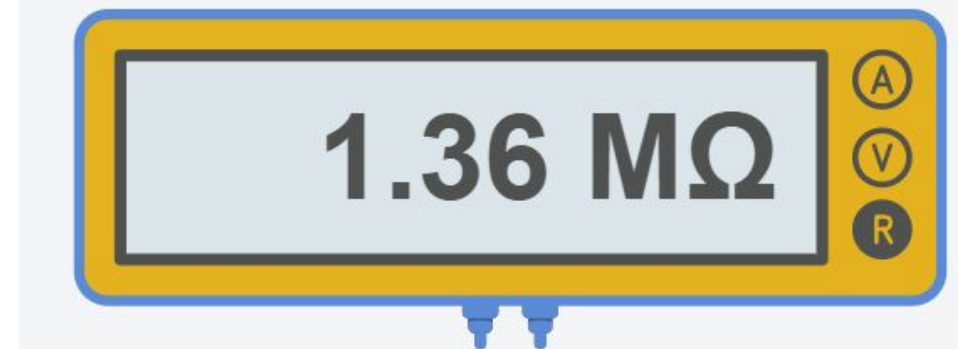
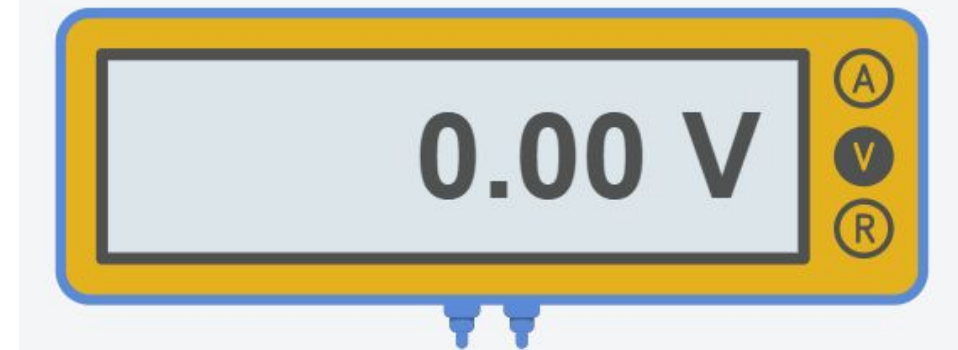
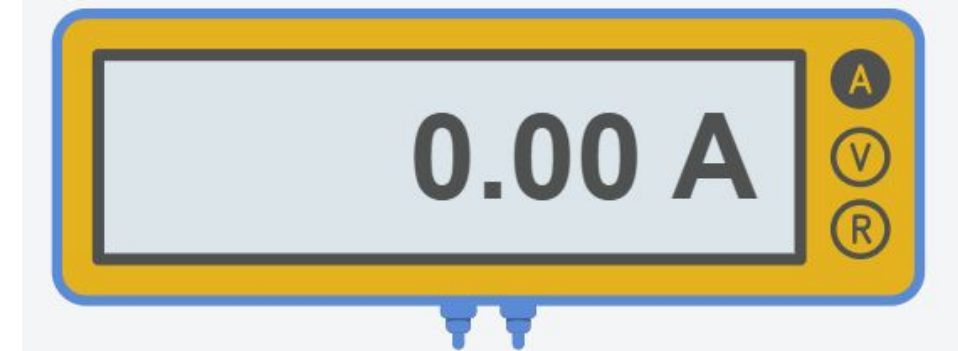
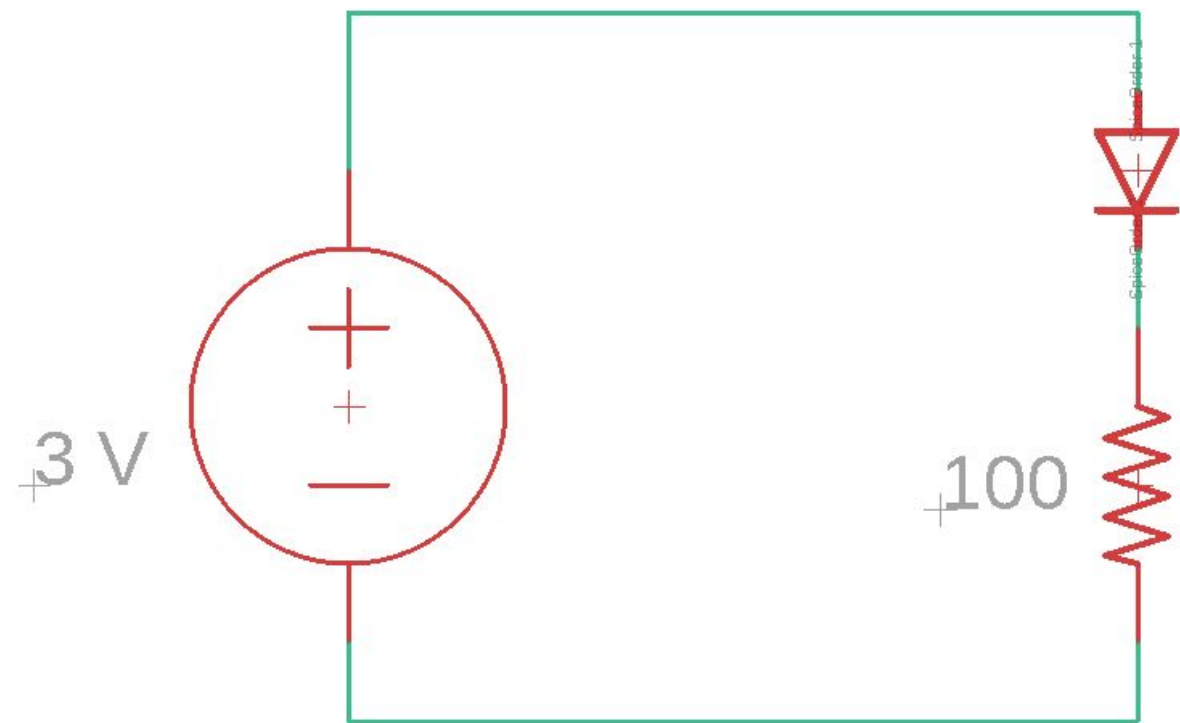
Espacio de Trabajo

En Tinkercad.

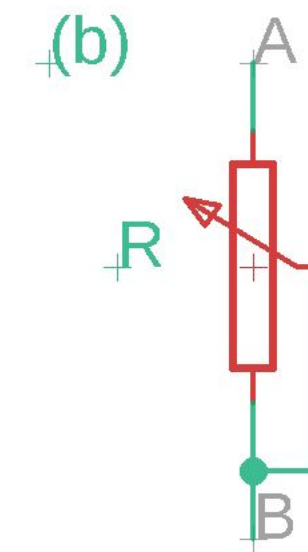
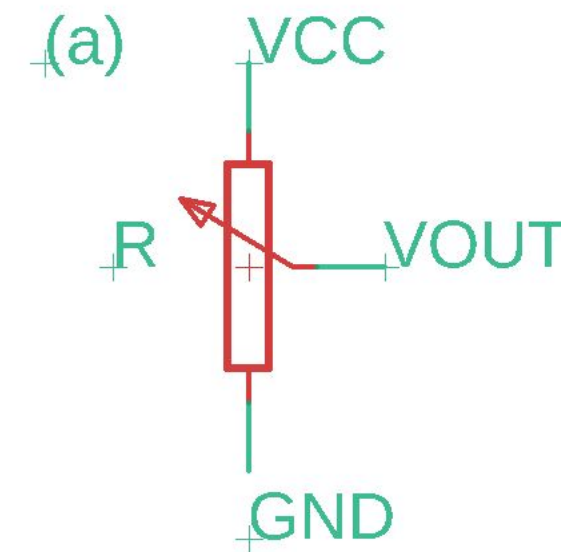
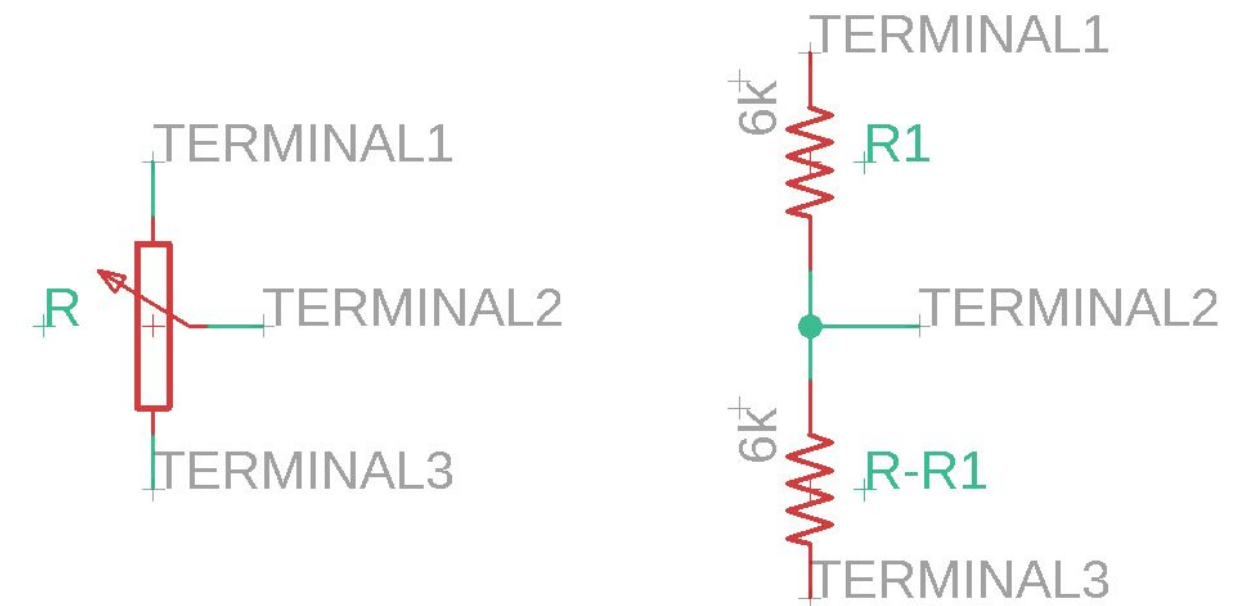
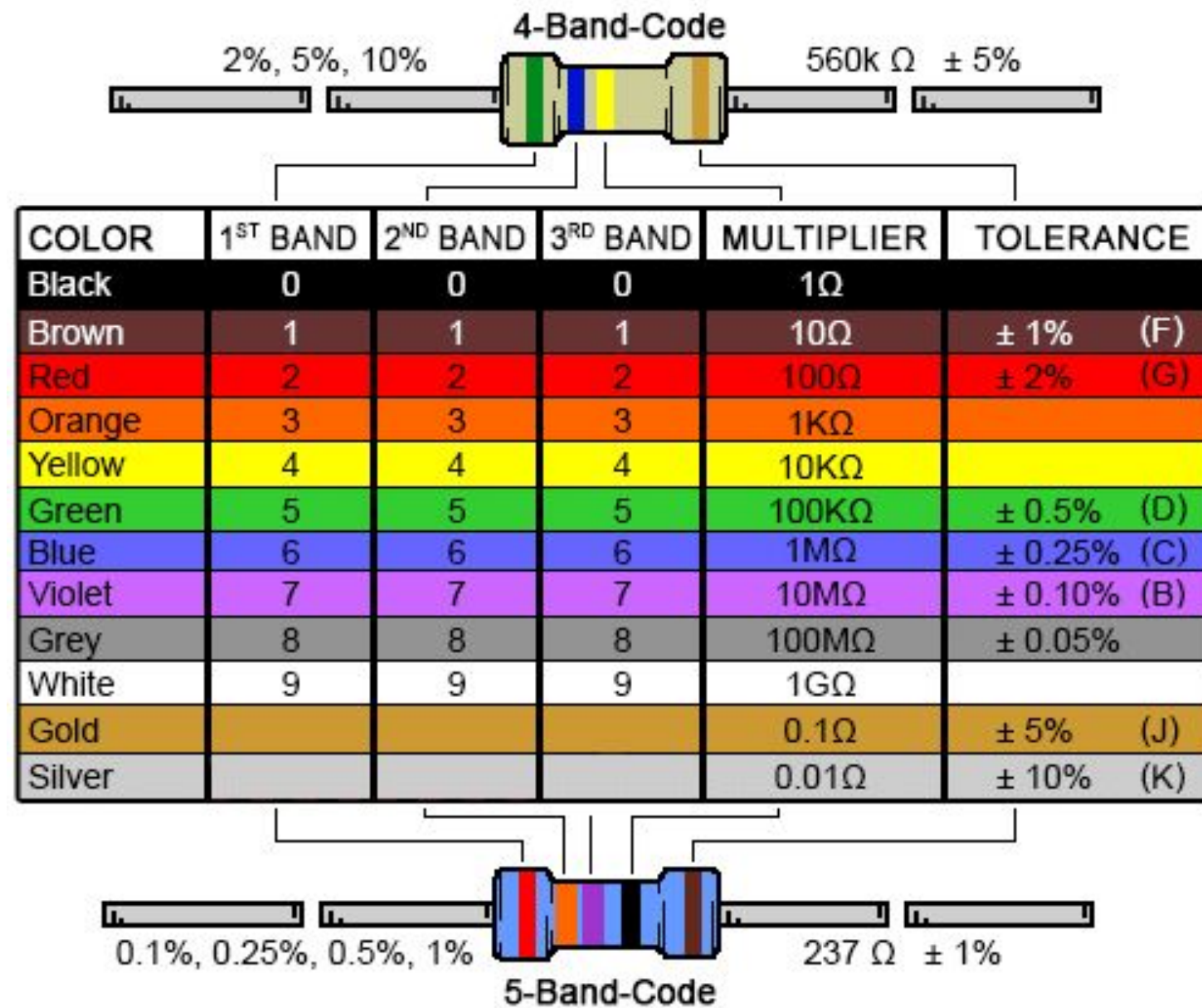


Simulación en Tinkercad

Encendido de LED y medición con herramientas.



Anexo: Resistencias



2.



Programación en Arduino *Tinkercad*

Introducción: Microcontroladores

¿Qué es un Microcontrolador?

Una unidad de microcontrolador es esencialmente una computadora pequeña en un solo chip. Está diseñado para administrar tareas específicas dentro de un sistema integrado sin requerir un sistema operativo complejo.

Estos circuitos integrados compactos (IC) contienen un núcleo de procesador (o núcleos), memoria de acceso aleatorio (RAM) y memoria de solo lectura programable borrable eléctricamente (EEPROM) para almacenar los programas personalizados que se ejecutan en el microcontrolador, incluso cuando la unidad está desconectada de una fuente de alimentación.



Microcontrolador PIC16F876A de Microchip



Introducción: Microcontroladores

¿Cuál es la diferencia con un microprocesador?

Un **microprocesador** (como los de una computadora) necesita componentes externos para funcionar, como memoria y dispositivos de entrada/salida.

Los **microcontroladores** integran periféricos de procesamiento, memoria y entrada/salida (E/S), incluidos temporizadores, contadores y convertidores de analógico a digital (ADC), en una unidad independiente eficiente y rentable.



**Microcontrolador PIC16F876A
de Microchip**



Microprocesador Intel Core i7



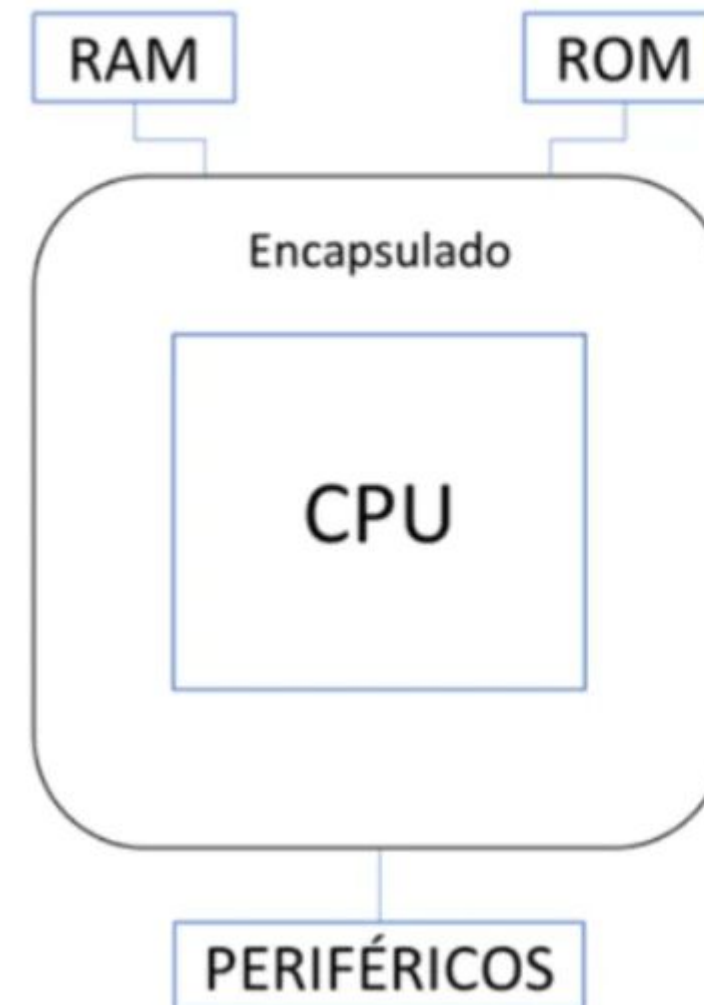
Introducción: Microcontroladores

¿Cuál es la diferencia con un microprocesador?

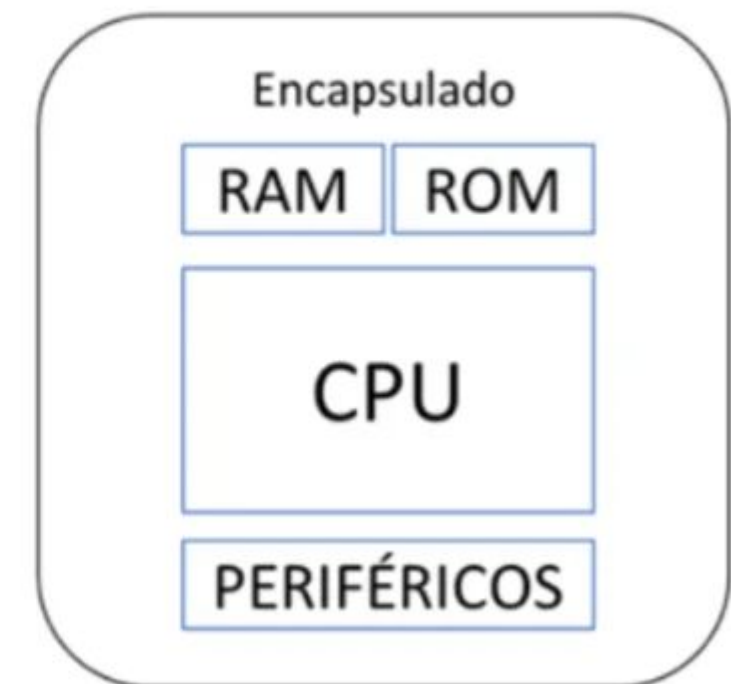
Un **microprocesador** (como los de una computadora) necesita componentes externos para funcionar, como memoria y dispositivos de entrada/salida.

Los **microcontroladores** integran periféricos de procesamiento, memoria y entrada/salida (E/S), incluidos temporizadores, contadores y convertidores de analógico a digital (ADC), en una unidad independiente eficiente y rentable.

MICROPROCESADOR



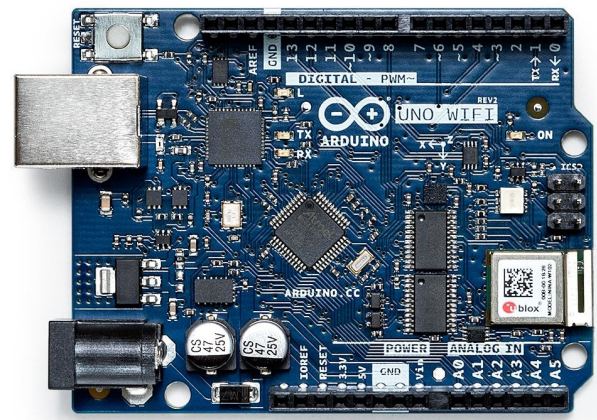
MICROCONTROLADOR



Esquema de Microprocesador vs. Microcontrolador

Introducción: Microcontroladores

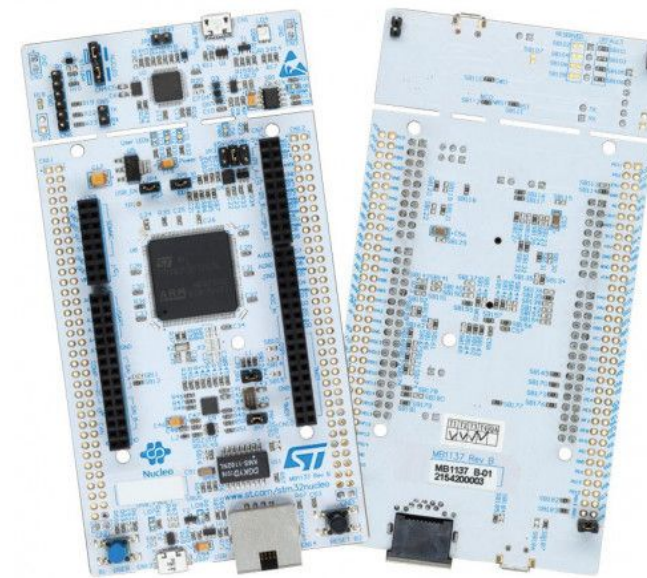
Diversas placas de desarrollo



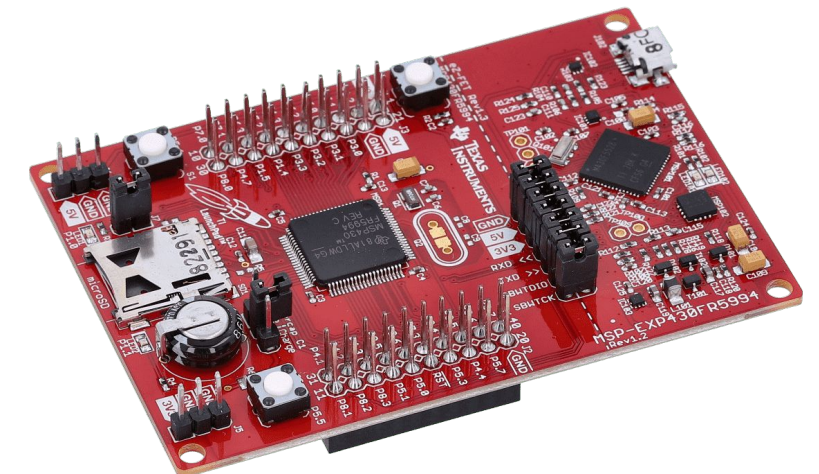
Arduino UNO



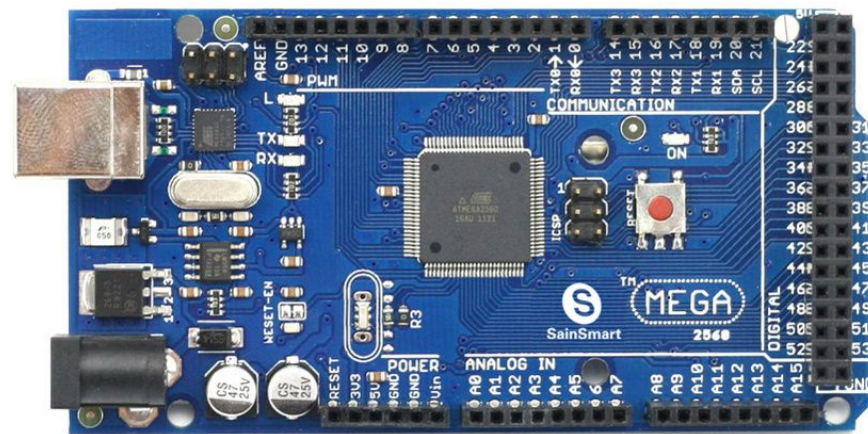
ESP32



STM Nucleo



MSP-ESP430G2ET (MSP430)



Arduino MEGA



Raspberry Pi



PIC18 Explorer



RAKWisblock (STM)

Introducción a Arduino

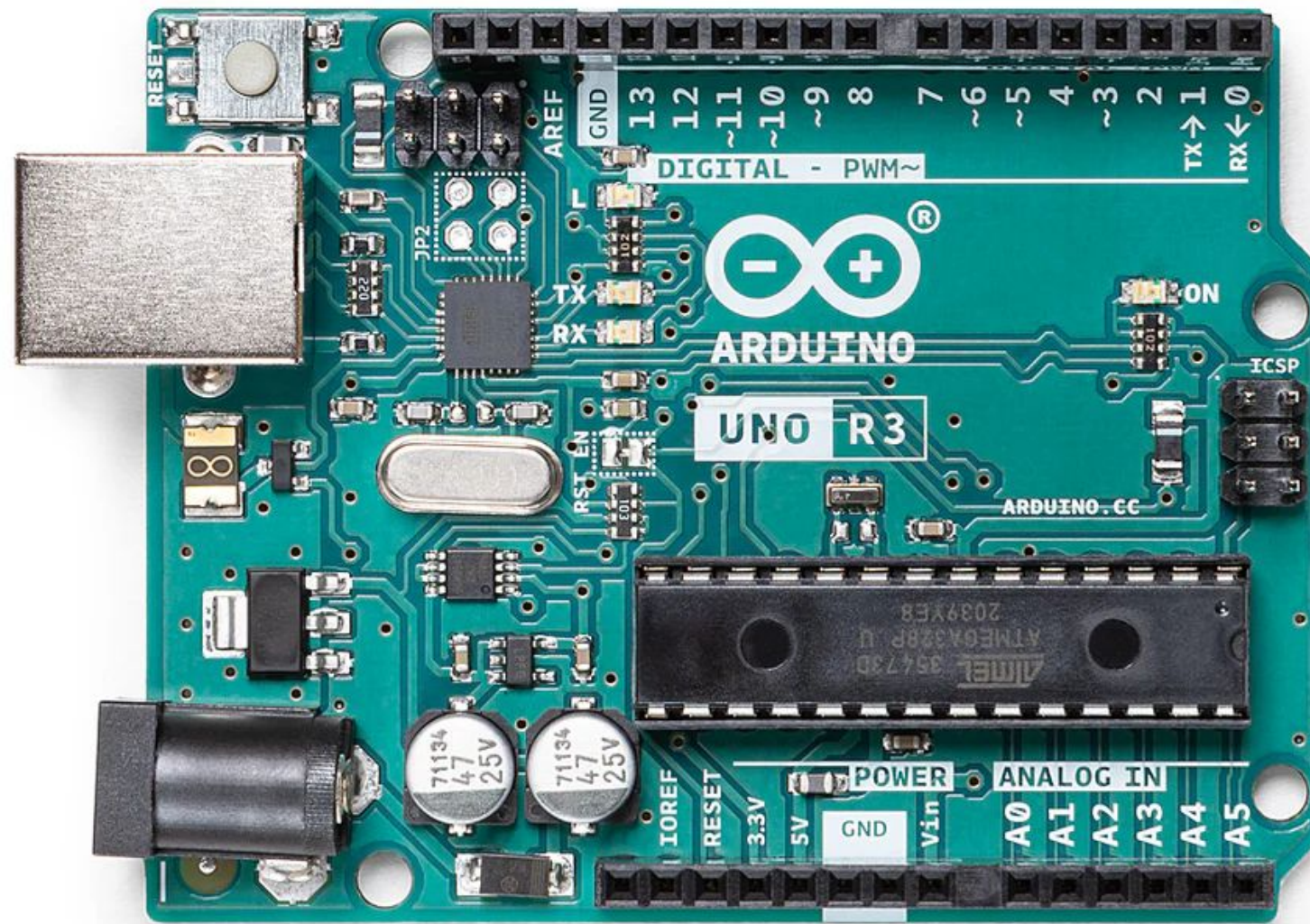
¿Qué es Arduino?

- Arduino es una plataforma electrónica de código abierto basada en hardware y software fáciles de usar.
- Es una plataforma de prototipado electrónico de código abierto que consta de hardware y software fácil de usar.
- El hardware de Arduino generalmente se refiere a la Breakout Board o conocido también como placa de desarrollo, que es una PCB que **incluye un microcontrolador y una serie de pines de entrada/salida** que permiten a los usuarios conectar diversos componentes electrónicos y sensores.
- El software de **Arduino IDE** consiste en un entorno de desarrollo integrado que facilita la escritura, carga y ejecución de código en la placa Arduino.

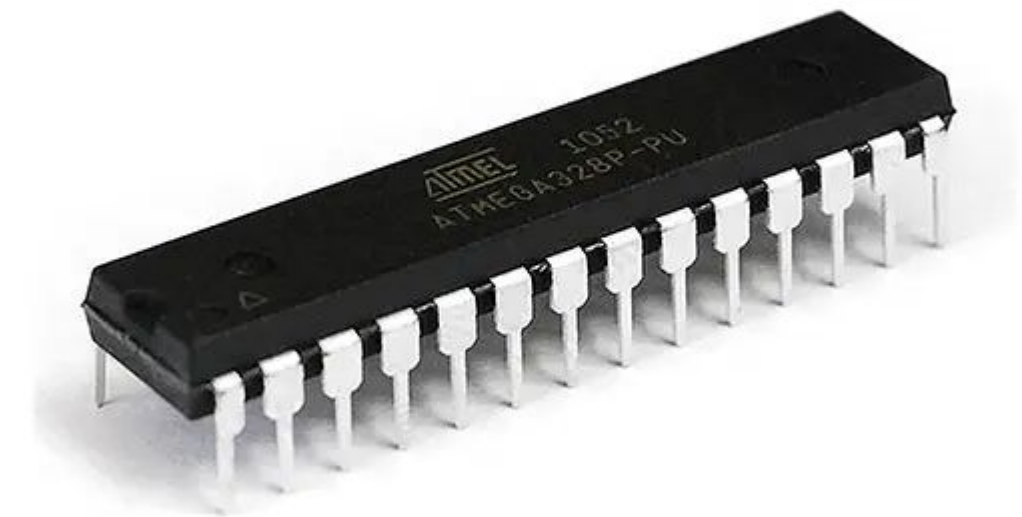


Introducción a Arduino

¿Qué es Arduino?



Arduino Uno Rev3



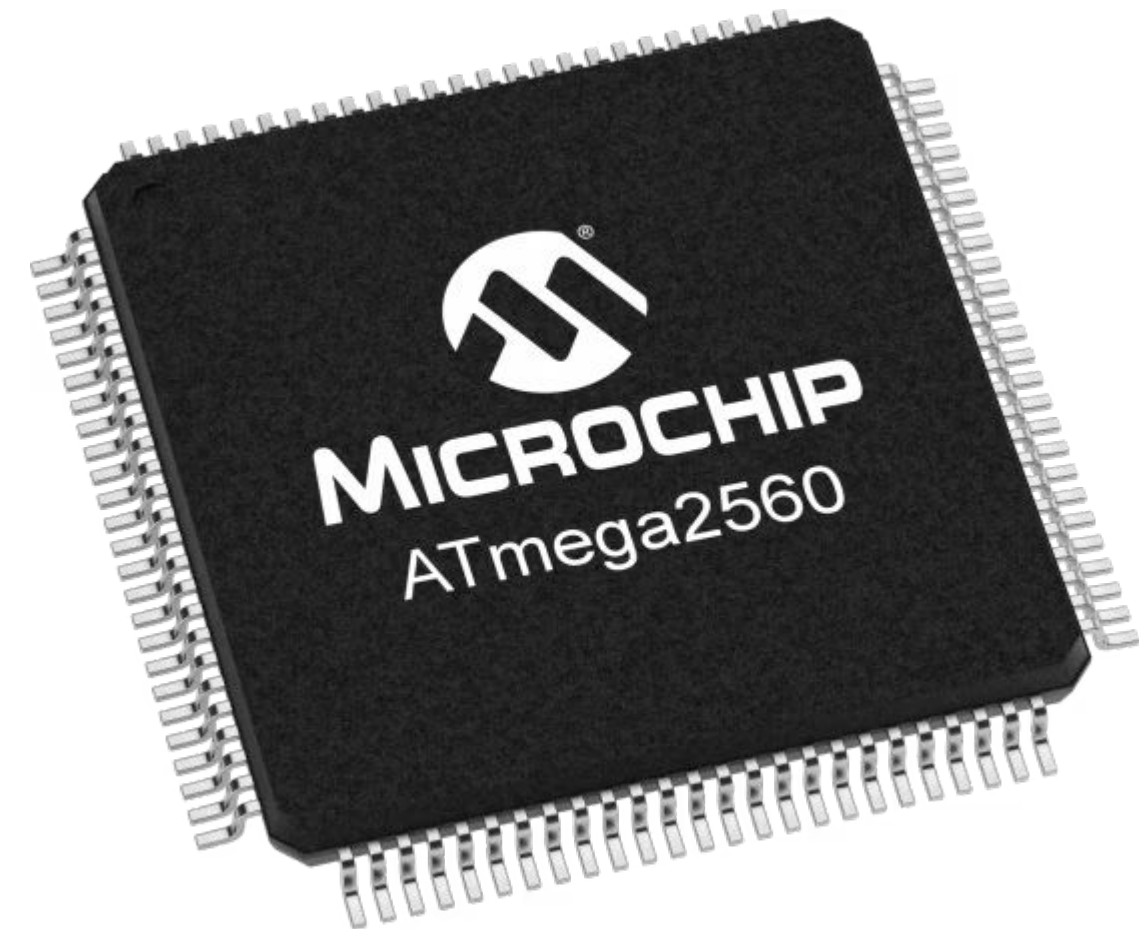
Microcontrolador ATMEGA328P

Introducción a Arduino

¿Qué es Arduino?



Arduino MEGA



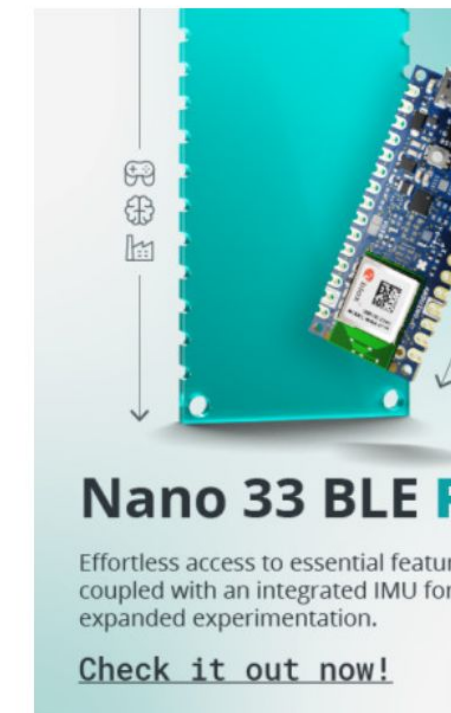
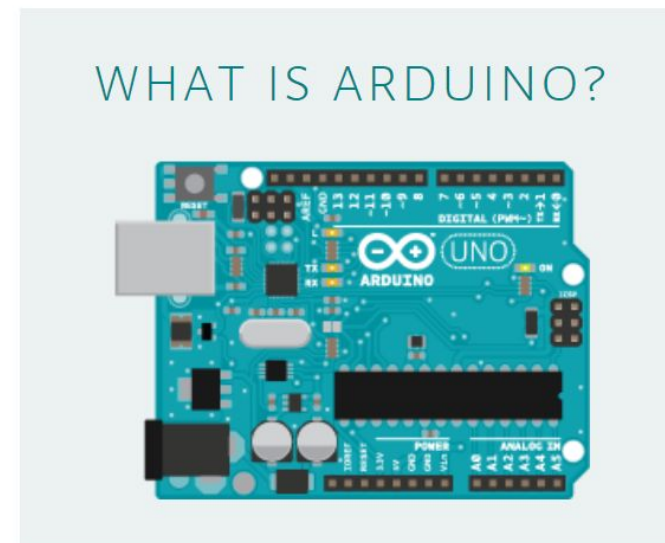
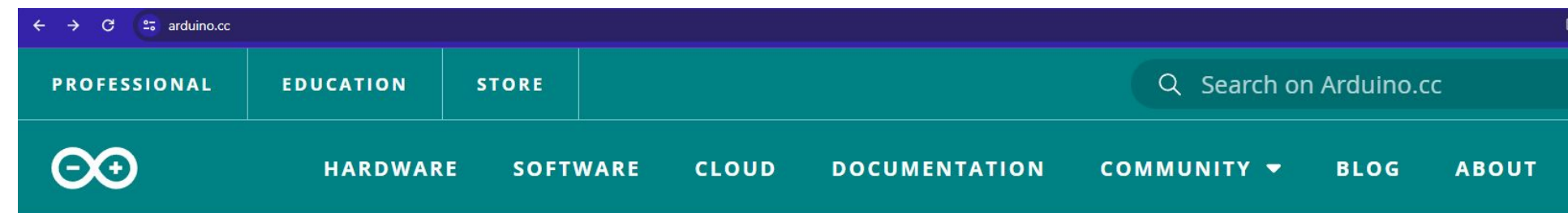
Microcontrolador ATMEGA2560



Introducción a Arduino

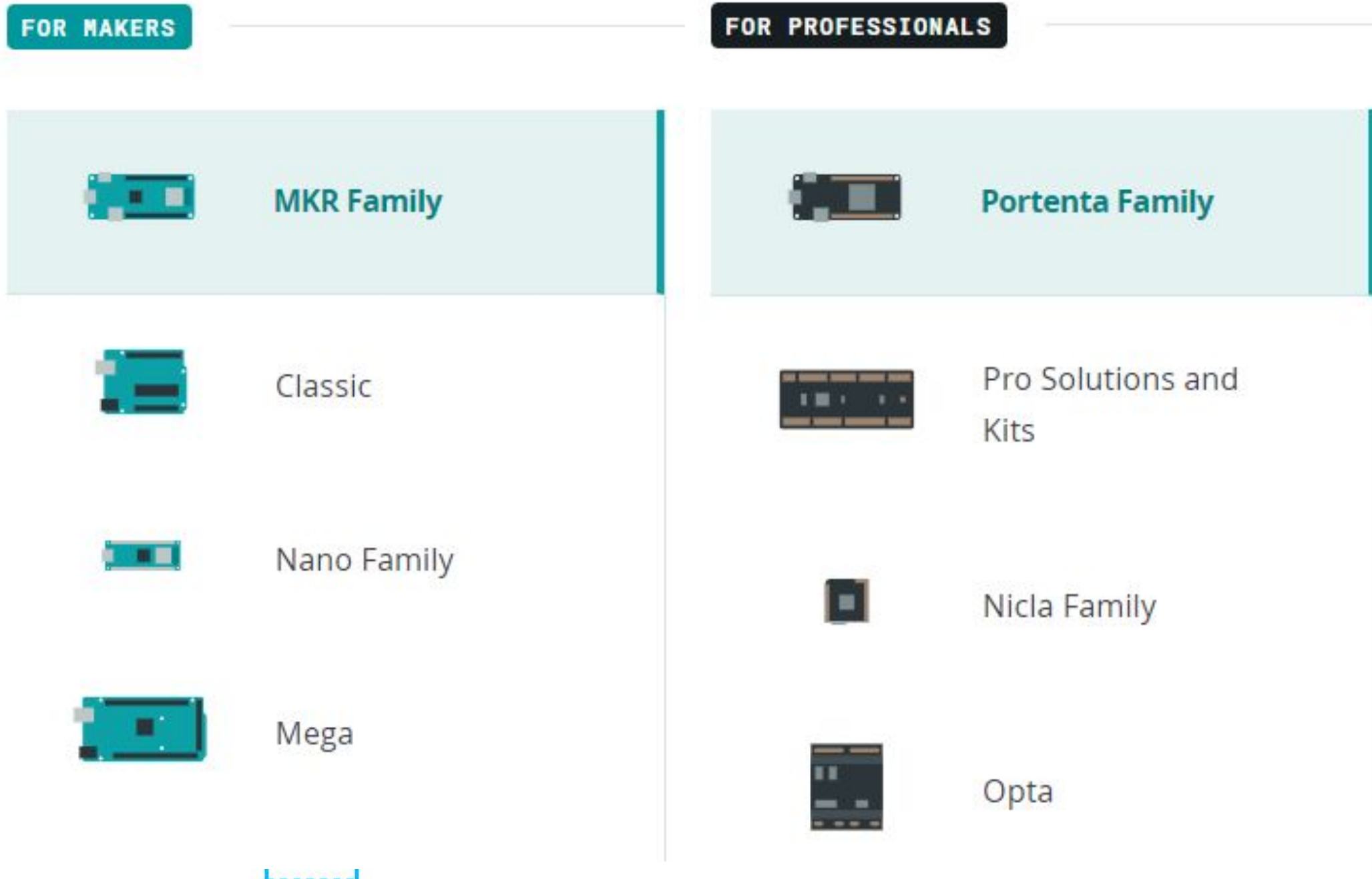
¿Qué es Arduino?

La página oficial de Arduino (www.arduino.cc) posee una gran cantidad de información tanto del hardware como del software, así mismo de tutoriales y librerías para una experiencia de desarrollo amigable.



Introducción a Arduino

Comparativa de placas

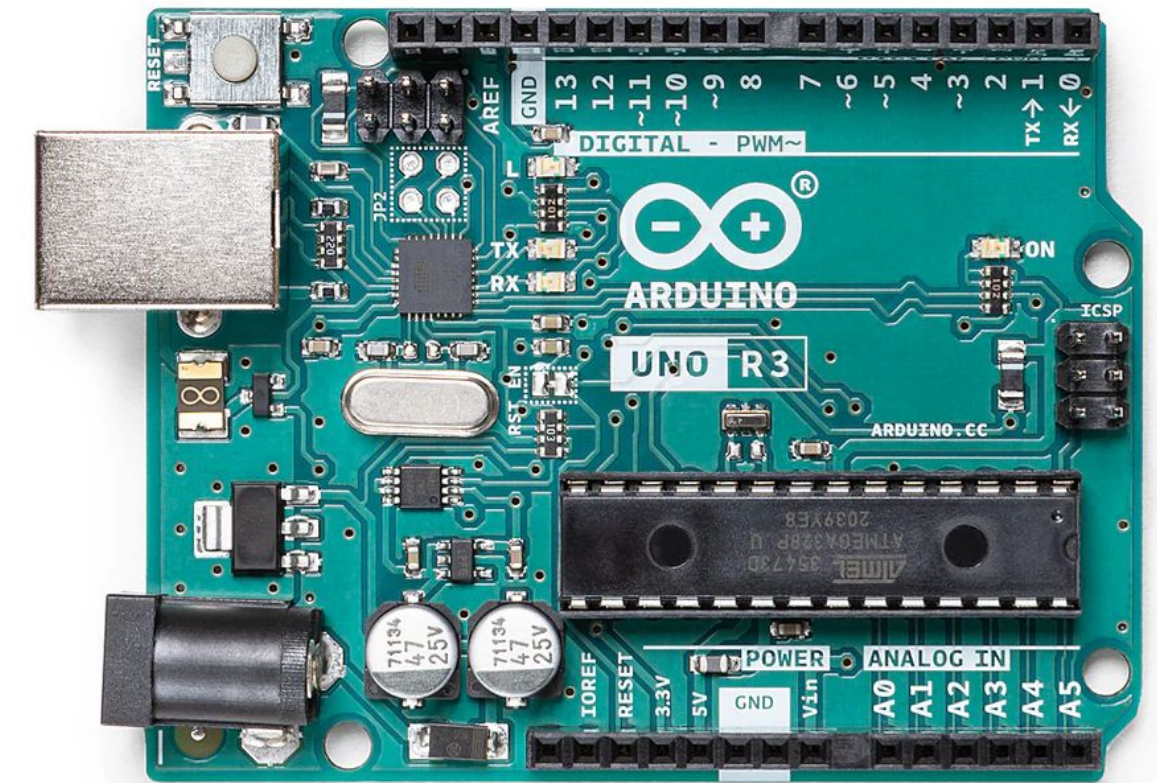


Arduino UNO R3

ATMEGA328P

El Arduino Uno R3 es una de las placas Arduino más populares y ampliamente utilizadas

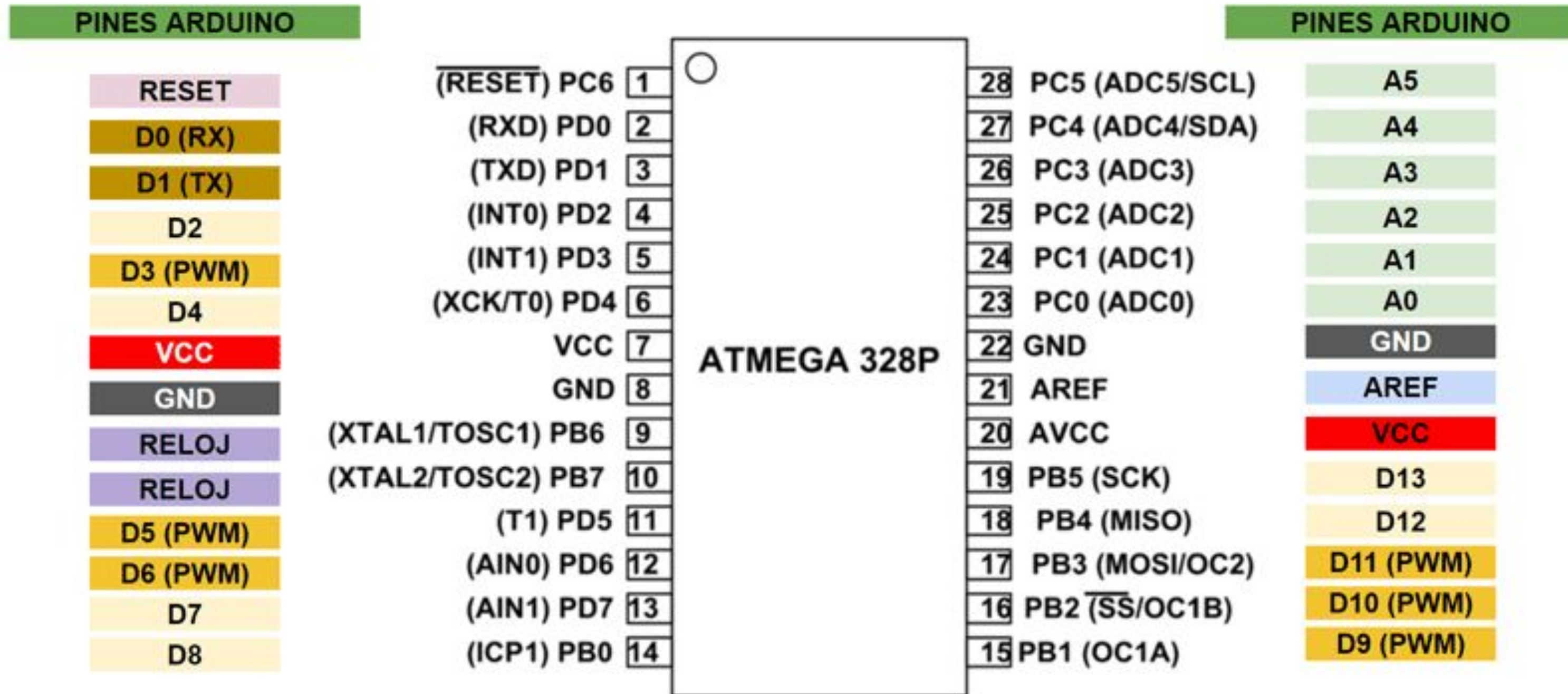
- Microcontrolador: Utiliza un microcontrolador ATmega328P.
- Pines de entrada/salida (E/S): Dispone de 14 pines digitales de entrada/salida, de los cuales 6 pueden ser utilizados como salidas PWM (Modulación por Ancho de Pulso) y 6 como entradas analógicas.
- Memoria: El ATmega328P tiene 32 KB de memoria flash (programa), 2 KB de SRAM y 1 KB de EEPROM.
- Velocidad de reloj: Opera a una velocidad de reloj de 16 MHz.
- Dimensiones: Aproximadamente 68.6 mm x 53.4 mm.



Arduino Uno Rev3

Arduino UNO R3

Pinout



Arduino MEGA

ATMEGA2560

- Microcontrolador: ATmega2560
- Velocidad de reloj: 16 MHz
- Voltaje de operación: 5 V
- Voltaje de entrada (límites): 6-20 V
- Pines digitales I/O: 54 (15 con salida PWM)
- Entradas analógicas: 16
- Memoria Flash: 256 KB (8 KB usados por el bootloader)
- SRAM: 8 KB
- EEPROM: 4 KB
- Interfaces de comunicación: 4 puertos UART, SPI, I2C
- Interrupciones externas: 6 pines
- Conversor A/D: de 10 bits

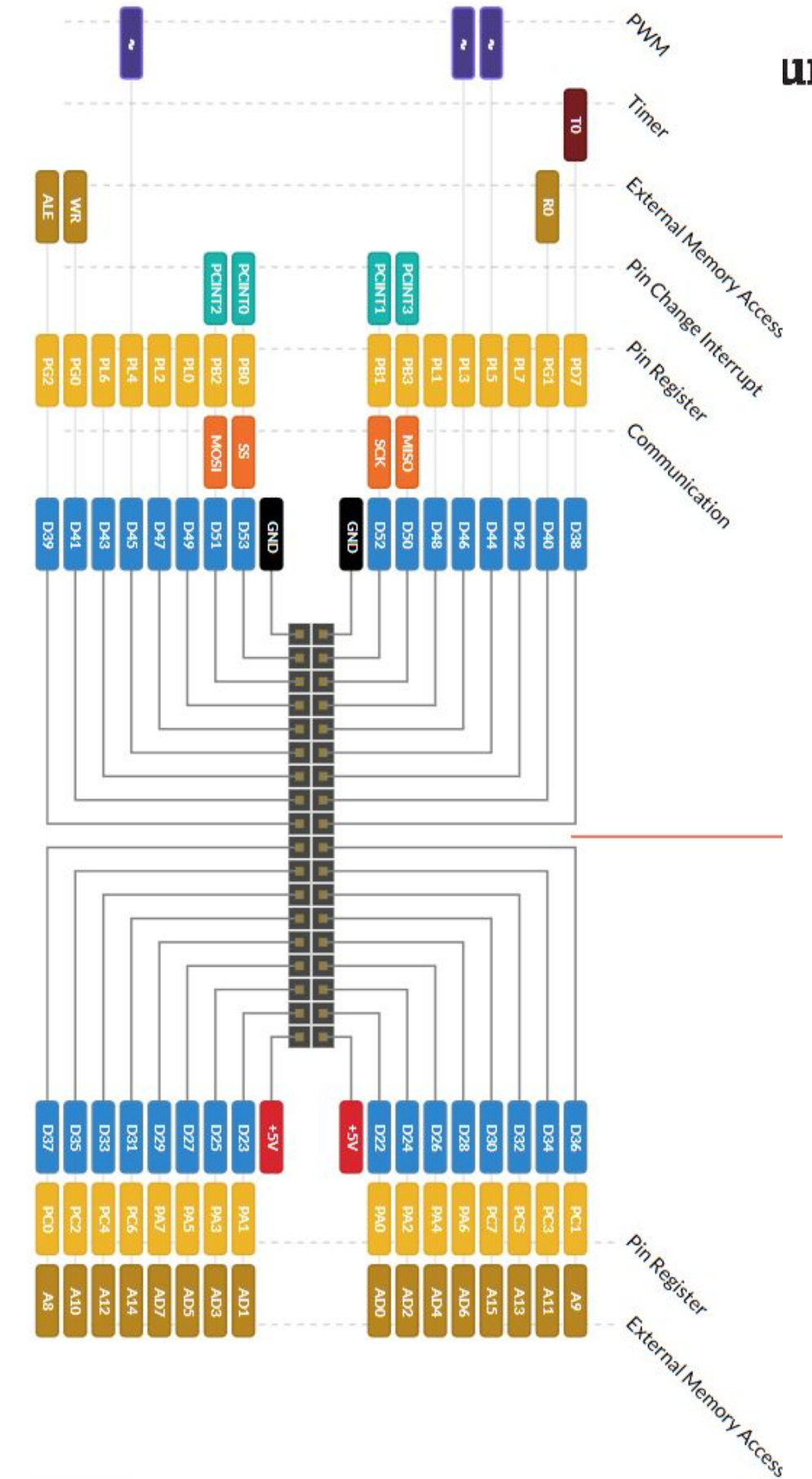
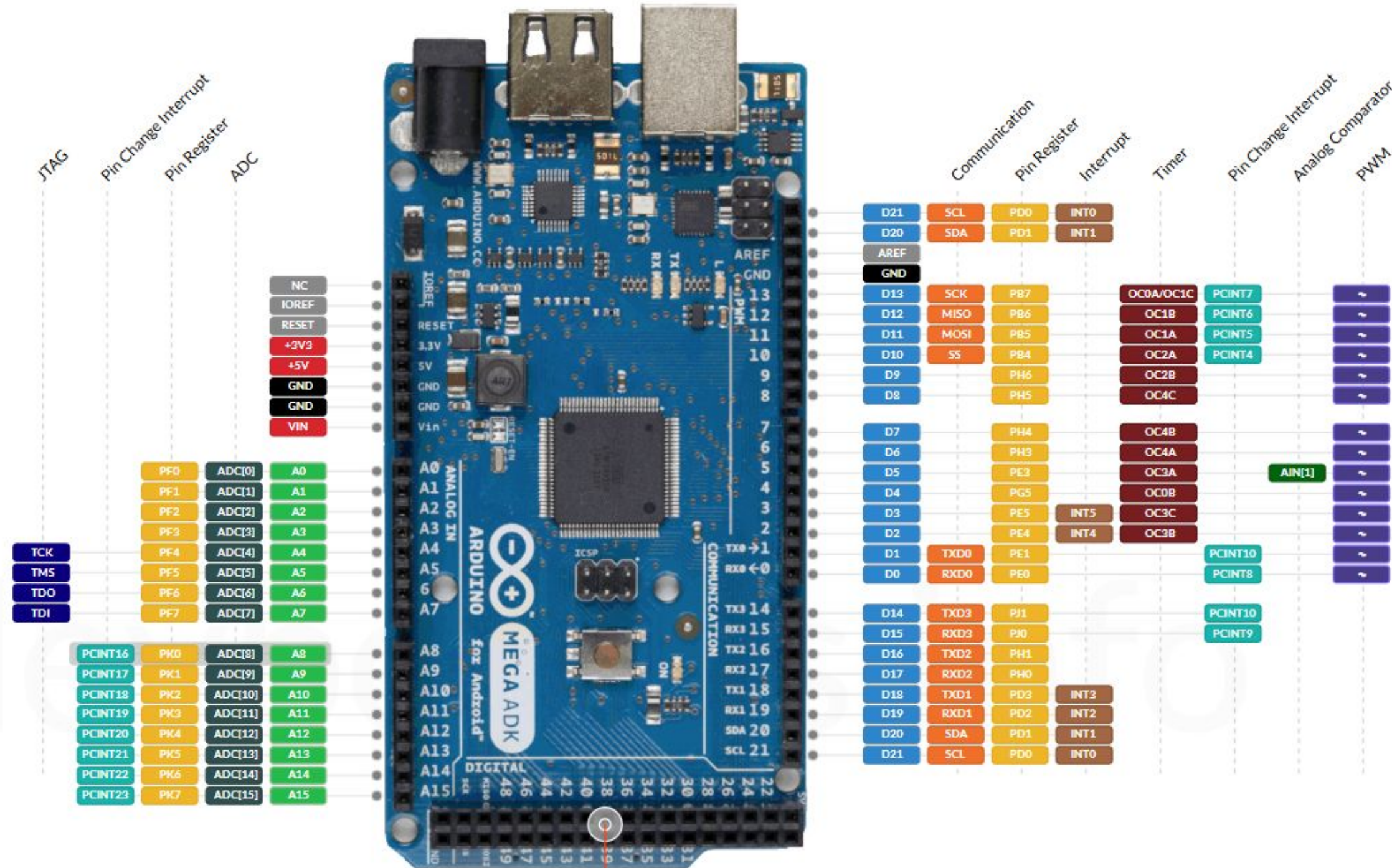


Arduino MEGA



Arduino MEGA

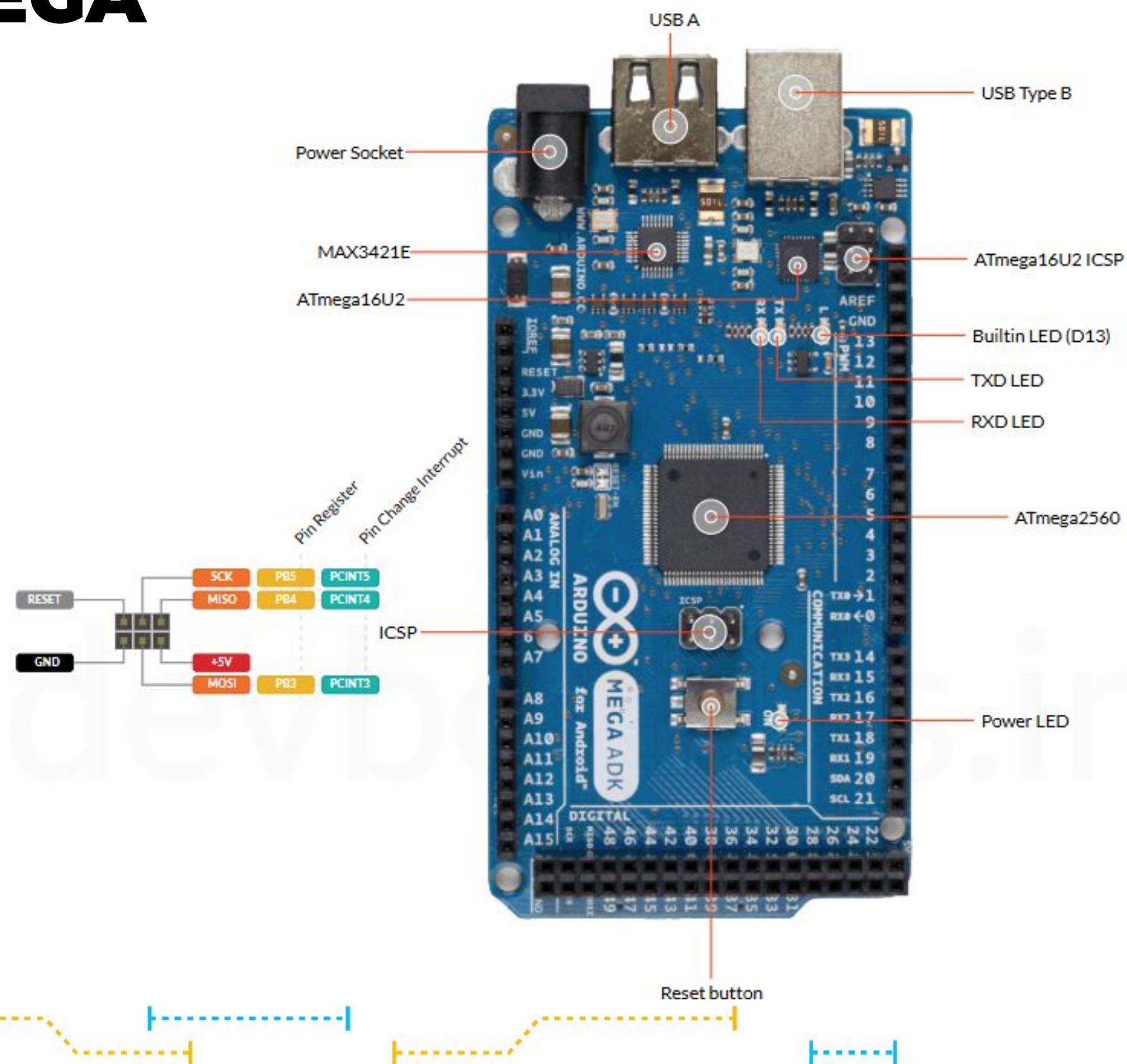
Pinout



undo <

Arduino MEGA

Pinout



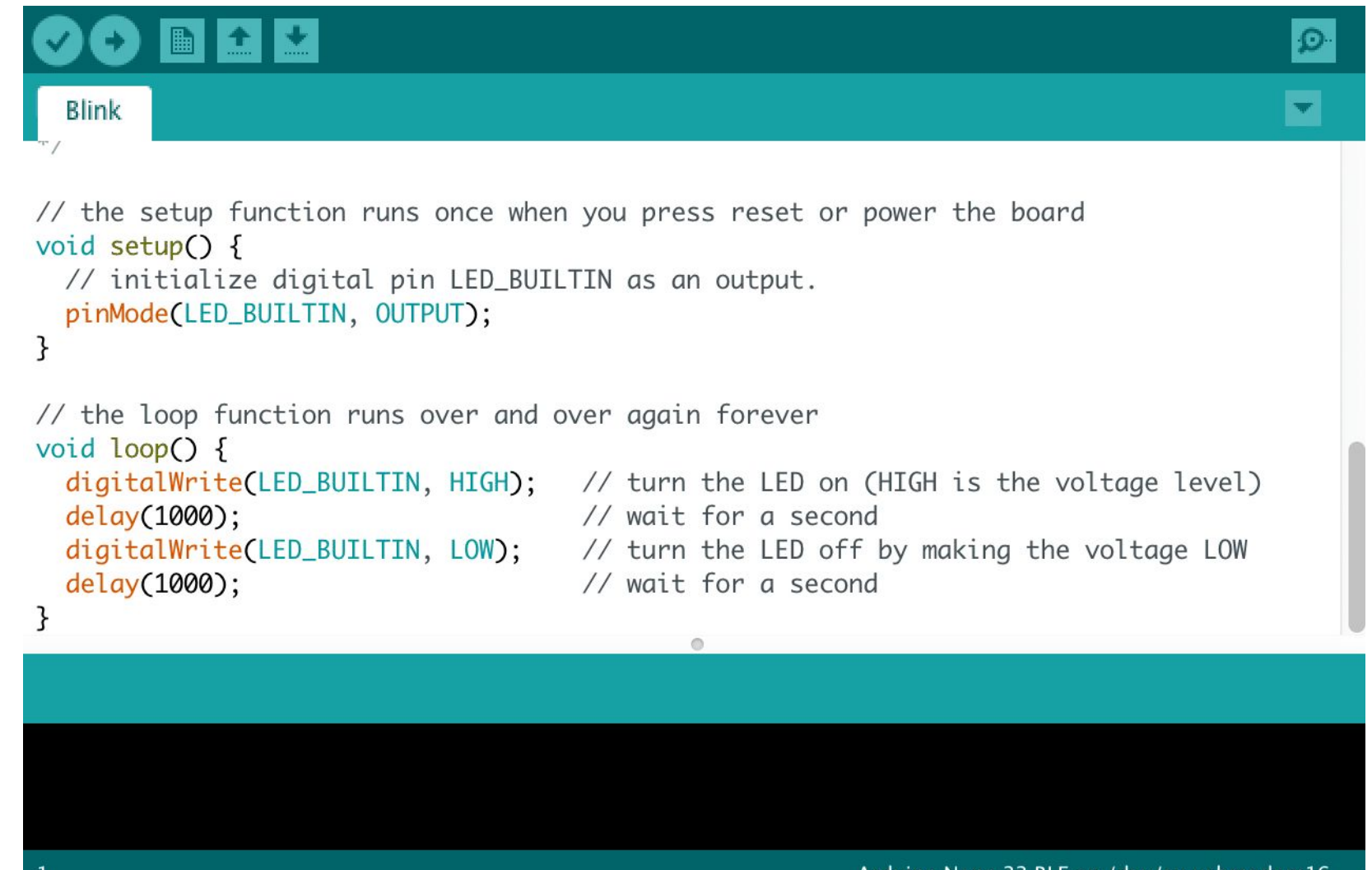
IDE de Arduino

Arduino IDE

El **IDE** de Arduino es una aplicación de software que permite escribir, compilar y cargar programas en placas Arduino.

Proporciona una interfaz simple para programar microcontroladores, ofreciendo herramientas como un editor de texto para el código, un compilador para traducir el código a instrucciones que el microcontrolador pueda ejecutar, y una interfaz para cargar ese código a través de un puerto USB.

El IDE de Arduino soporta varios lenguajes de programación, aunque comúnmente se utiliza un subconjunto de C/C++.



```

// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000);                     // wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off by making the voltage LOW
  delay(1000);                     // wait for a second
}

```

Captura de IDE de Arduino



IDE de Arduino

Instalación

Dirigirse al sitio web oficial de Arduino (<https://www.arduino.cc/en/software>) y descarga el IDE de Arduino. Podrás ver las múltiples versiones disponibles para Windows, Linux y Mac OS X.



Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits

Windows MSI installer

Windows ZIP file

Linux Appliance 64 bits (X86-64)

Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits

macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

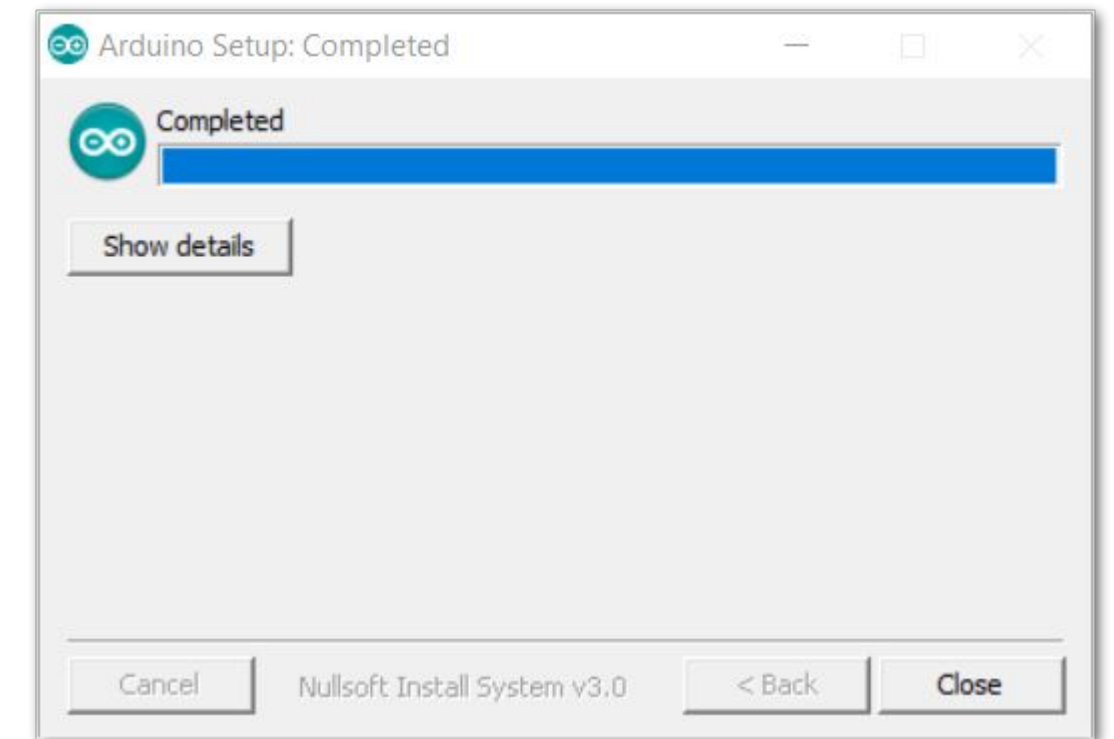
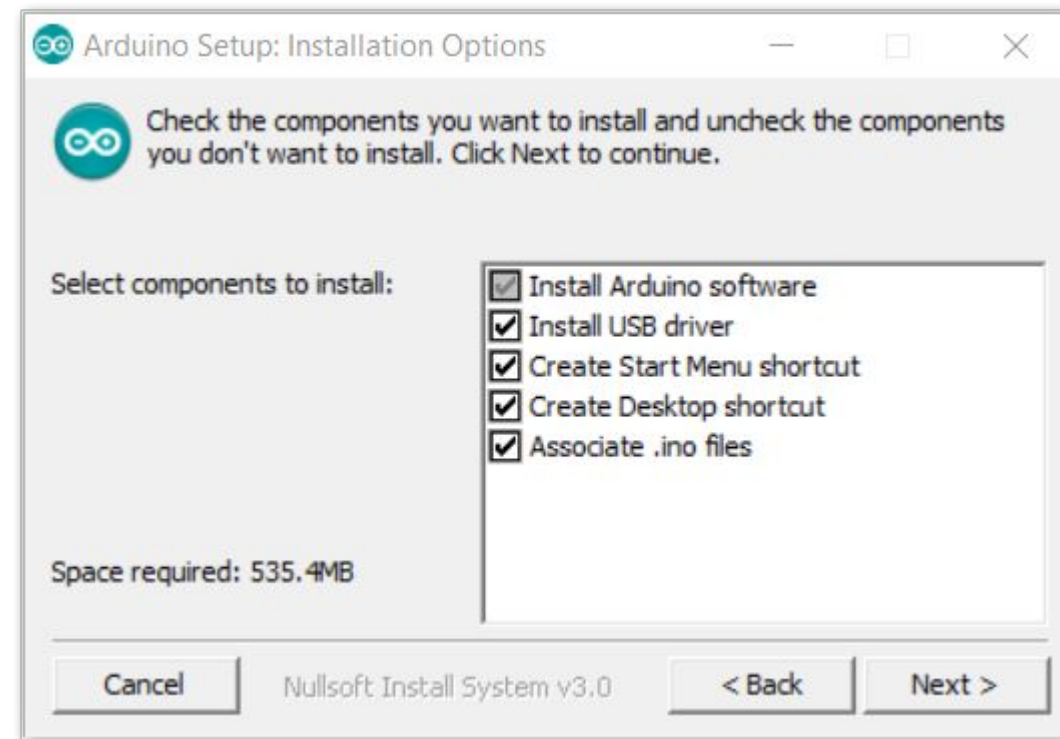
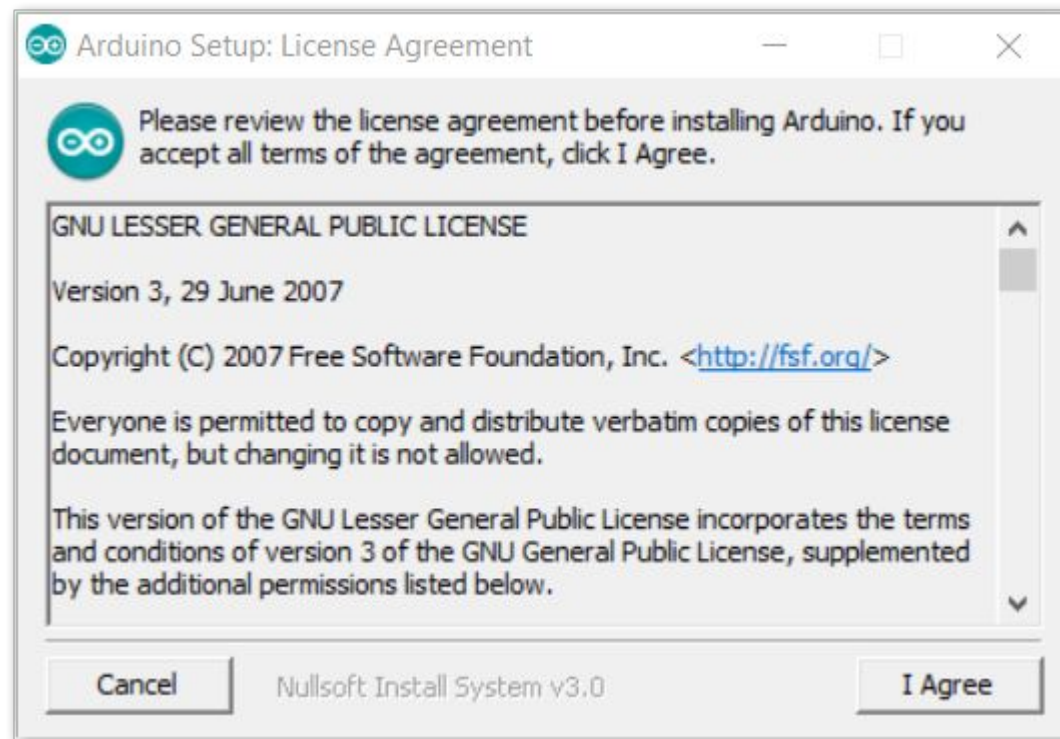
[Release Notes](#)

? Help

IDE de Arduino

Instalación

Aceptar todos los permisos.



IDE de Arduino

Guía de la interfaz

Guía de la interfaz Arduino IDE

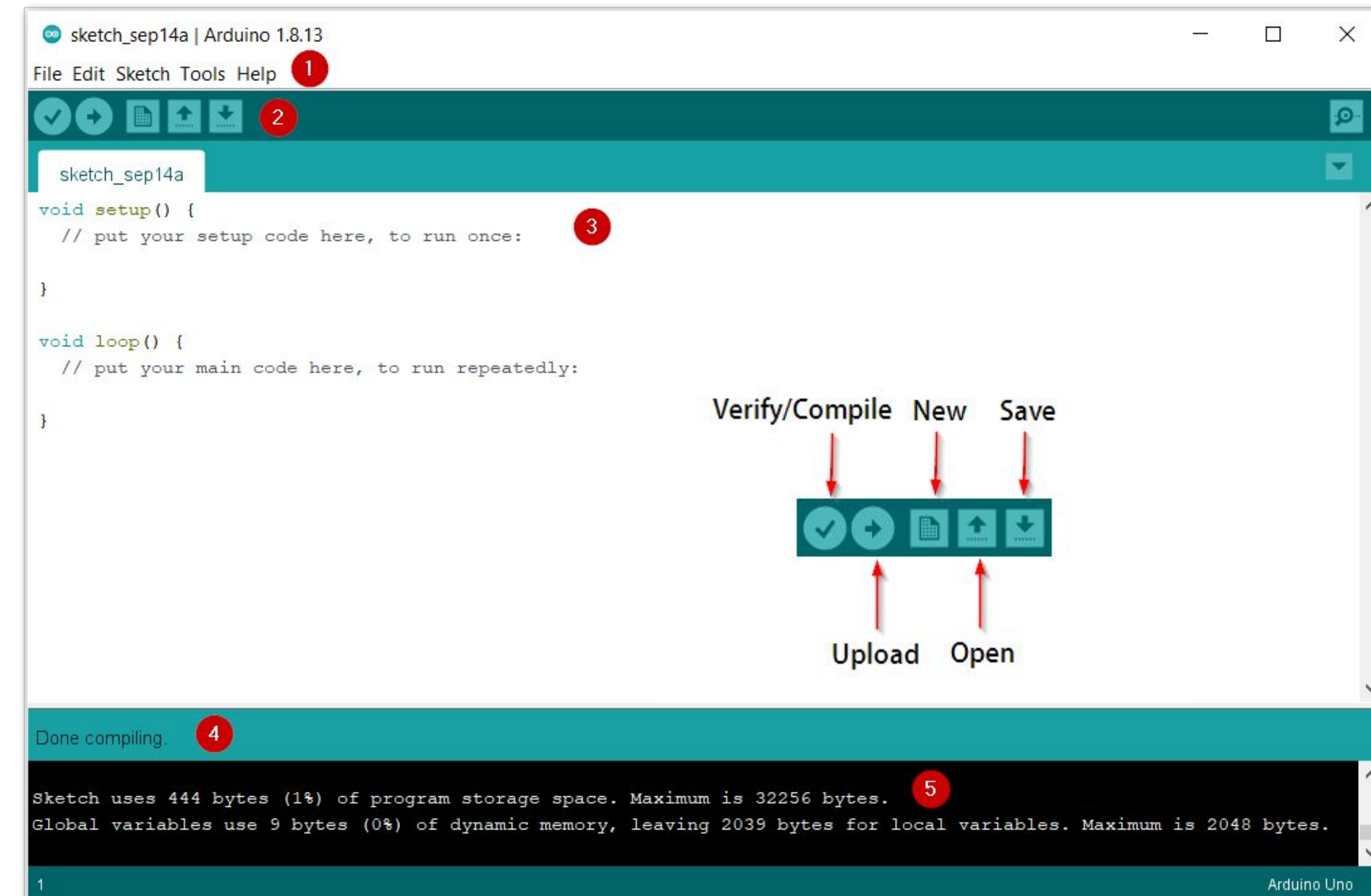
1. Menú de opciones del IDE

Puede configurar algunos parámetros generales como el puerto serie, la información de la placa, las bibliotecas, los parámetros de edición, etc.

2. Botones de operación

Los botones de operación tienen cinco operaciones:

- Verificar/Compilar el código fuente;
- Cargar el código compilado;
- Abrir una nueva ventana del IDE Arduino
- o una aplicación existente;
- Guardar la aplicación actual.



IDE de Arduino

Guía de la interfaz

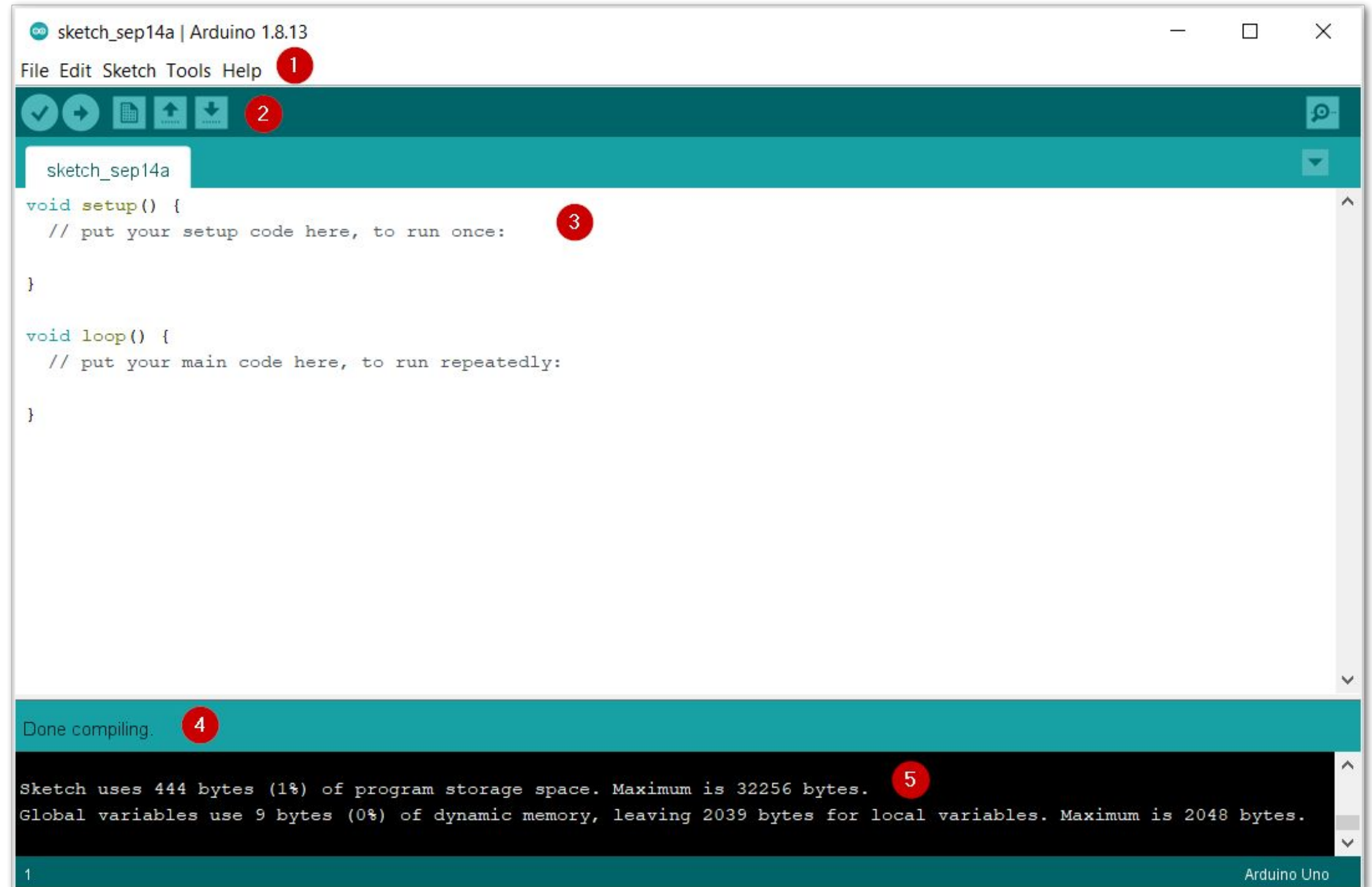
3. Área de código

Puedes editar el código fuente, que será compilado y cargado en WisBlock más tarde en esta área.

4. Área de estado

5. Área de Mensaje de Salida

Puedes ver el mensaje de salida en esta área, ya sea información de fallo o de éxito.



IDE de Arduino

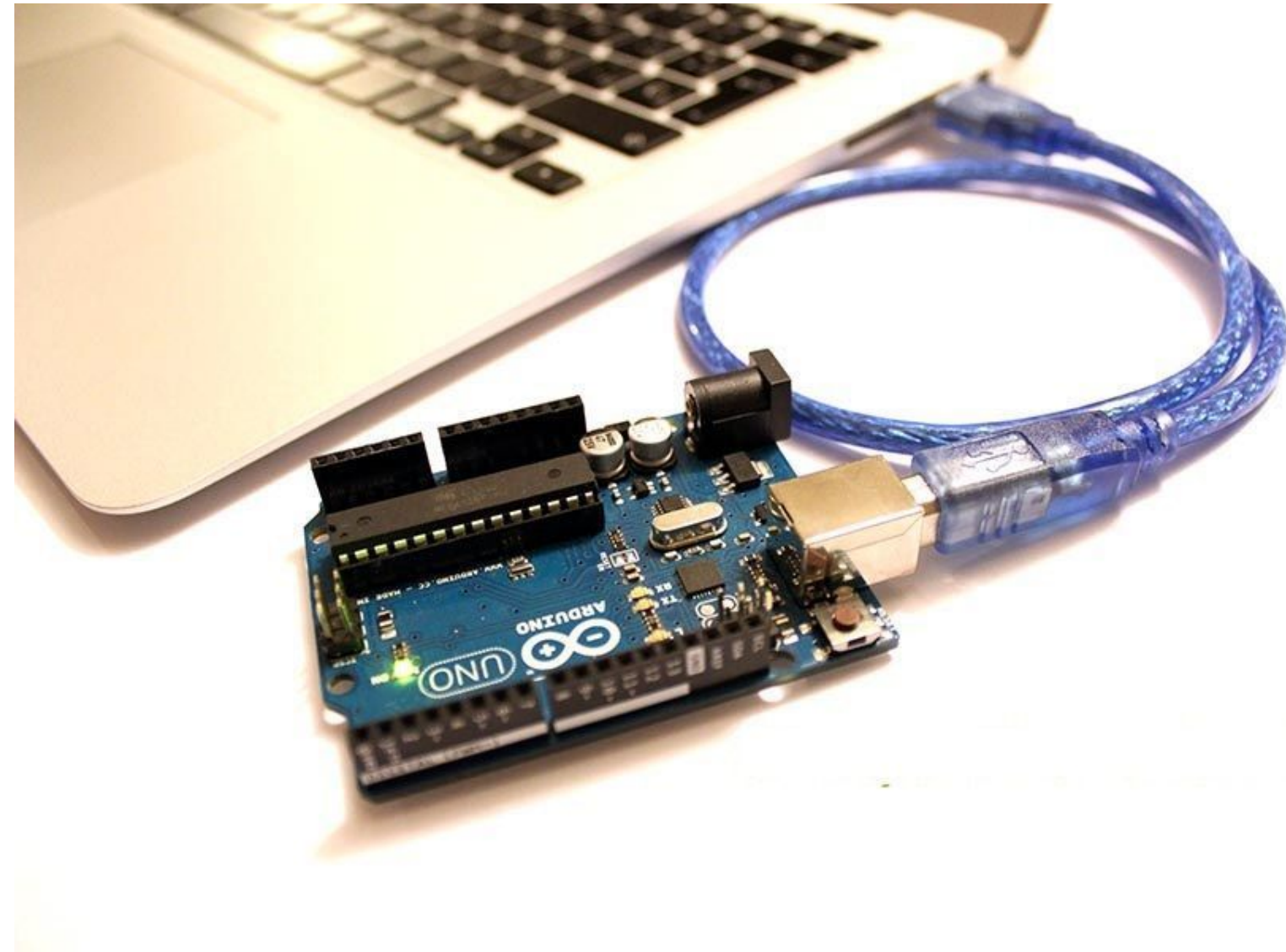
Guía de la interfaz



IDE de Arduino

Conexión con la placa

1. Conecta el Arduino a tu computadora: Utiliza el cable USB para conectar tu Arduino a uno de los puertos USB disponibles en tu computadora. Asegúrate de que la conexión esté segura y estable.



IDE de Arduino

Conexión con la placa

2. Abre el software de Arduino: Una vez que hayas conectado tu Arduino, abre el software de Arduino en tu computadora. El software debería detectar automáticamente la conexión y seleccionar el puerto COM correcto.

3. Configura el tipo de placa: En el menú «Herramientas» del software de Arduino, selecciona el tipo de placa que estás utilizando. Por ejemplo, si estás utilizando un Arduino Uno, elige «Arduino/Genuino Uno» en la opción «Placa».

4. Selecciona el puerto COM: En el mismo menú «Herramientas», selecciona el puerto COM al que está conectado tu Arduino. Si no estás seguro de cuál es el puerto correcto, puedes verificarlo en el Administrador de dispositivos de tu computadora.



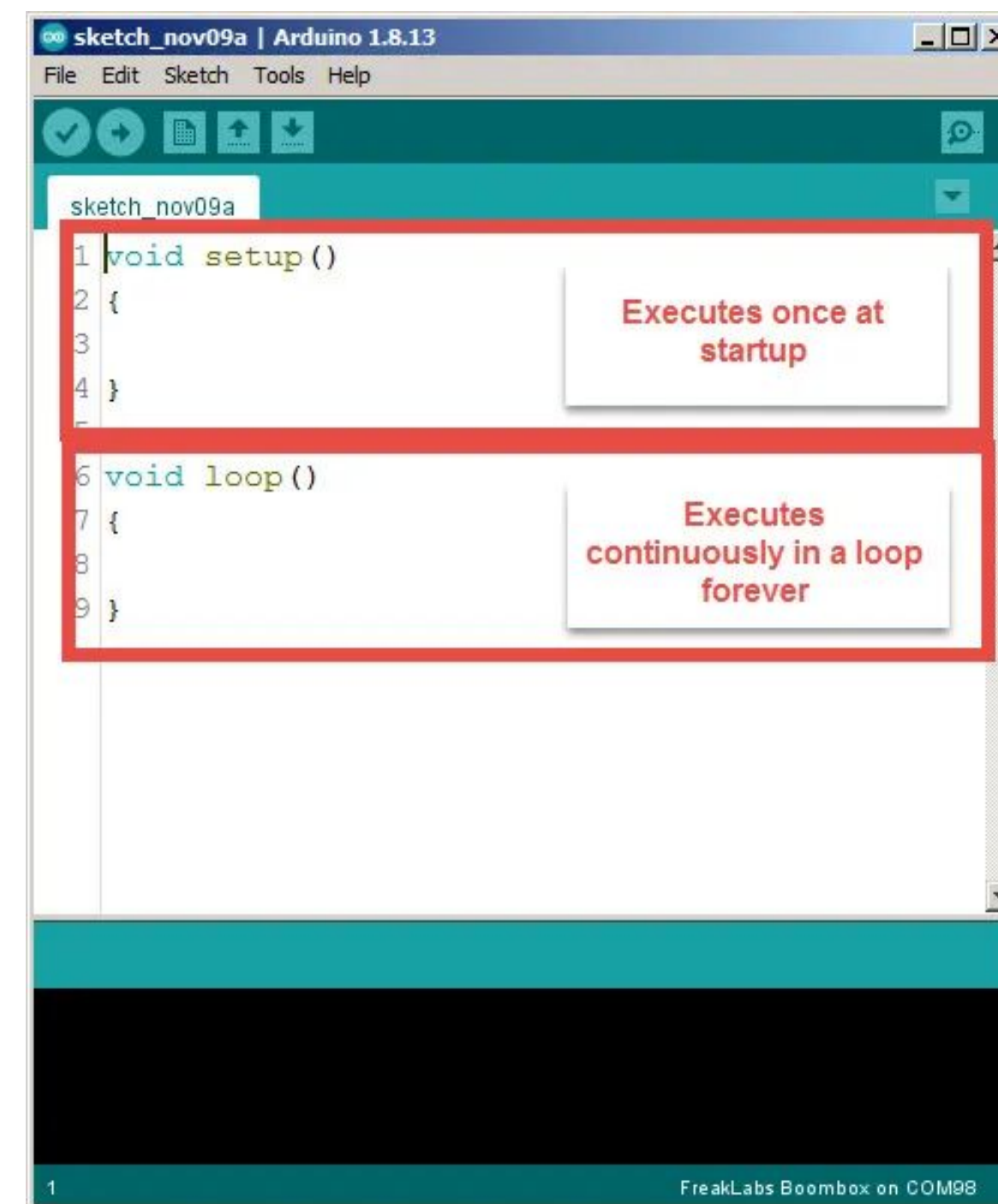
IDE de Arduino

Estructura

La estructura básica del código en Arduino IDE consta de dos partes principales:

- **setup():** Función que se ejecuta una sola vez al iniciar el programa. Se usa para inicializar configuraciones, como definir pines o iniciar comunicaciones.
- **loop():** Función que se ejecuta de manera continua después de setup(). Aquí se coloca el código que debe ejecutarse repetidamente, como la lectura de sensores o el control de actuadores.

Ambas funciones son obligatorias para que el programa funcione en Arduino.



```

1 void setup()
2 {
3
4 }
5
6 void loop()
7 {
8
9 }
  
```

Executes once at startup

Executes continuously in a loop forever

Estructura



GRACIAS

Internet de las Cosas 2025-I

